

几何造型技术



——研究在计算机中如何表示物体模型形状的技术。

几何造型系统中，有三种描述物体的三维模型：线框、曲面和实体模型。

Curves and Surfaces

——曲面模型能够表达复杂的雕刻曲面，在几何造型中具有重要地位。

Bezier、B-Spline曲线曲面适用于表达工业造型中较光滑曲线曲面。

- **Parametric curves and surface: Some Concepts**
- **Bezier curves and surfaces**
- **B-Spline curves and surfaces**

Parametric curves and surface

- In Graphics, we usually design a scene and then generate realistic image(真实感图像) by using rendering equation.
- It's necessary to introduce **geometric modeling (几何造型)** for scene design.
- **How to represent 3D shapes (models) in computer?**



History of Geometric Modeling (几何造型)

- Surface Modeling (曲面造型) :
 - In 1962, **Pierre Bezier**, an engineer of French Renault Car company, propose a new kind of curve representation, and finally developed a system UNISURF for car surface design in 1972.



Bezier Curve / Pierre Bezier

[vBack to Last Page](#) > [vFull Glossary](#)

Definition: A Bezier curve is a curved line or [path](#) defined by mathematical equations. It was named after Pierre Bezier^ a French mathematician and engineer who developed this method of computer drawing in the late 1960s while working for the car manufacturer Renault. Most graphics software includes a pen tool for drawing paths with Bezier curves. (Continued below...)

The most basic Bezier curve is made up of two end points and [control handles](#) attached to each [node](#). The control handles define the shape of the curve on either side of the common node, Drawing Bezier curves may seem baffling at first; it's something that requires some study and practice to grasp the geometry involved. But once mastered^ Bezier curves are a wonderful way to draw!

Pierre Bezier was born September 1, 1910 and died November 25^ 1999 at the age of 89. In 1985 he was recognized by ACM SIGGRAPH with a '[Steve d, Ceens' mwmrd](#) for his lifetime contribution to computer graphics and interactive techniques.



A Bezier curve with three nodes. The center node is selected and the control handles are visible.



- Solid Modeling （实体造型）：
 - In 1973, Ian Braid of Cambridge University developed a solid modeling system for designing engineering parts.
 - Ian Braid presented in his dissertation “Designing with volumes”, this work being demonstrated with the BUILD-1 system.



Solid.JVLocleling.org

LLThe website for Solid Association.

Home

^elcorae

Sy mposia.

^Current

People

Bezier Award Who's Who
Join SMA.

Business

Executive* ^oard.

Pr oc edures

SpO MRCE

Ian Braid, Alan Grayer and Charles Lang

The 2008 Pierre B^9ier Award Recipients

This group of three people made many fundamental contributions to practical solid modelling, and their work has had a profound influence on today's commercial solid modelling systems. They commenced working together in the CAD Group at

the Computer Laboratory, Cambridge University. The Group was set up by Charles Lang under Prof Maurice Wilkes

direction in 1965 to undertake research on tools for building mechanical CAD/CAM systems, with an emphasis on software system components, computer graphics and computational geometry. Initial experiments in solid modelling were made in 1969. Also in 1969 Ian Braid joined the Group where, under Charles Lang's supervision, he developed the BUILD boundary representation modeller, the most advanced such system of its day. Whereas other systems used faceting to avoid time problems of calculating intersections between non-planar surfeces, the BUILD team tackled such problems head-on. Ian was awarded his PhD in 1973. Alan Grayer joined the group in 1971 and, also under Charles^ supervision, developed algorithms for the automatic machining of prismatic parts modelled in BUILD. These were machined on a model making machine, built by

itie Group in 1971 following an inspirational visit to B 伐 ier at Renault in Paris. Alan was awarded his PhD in 1977. Ian

linen developed a completely new solid modeller, BUILD 2, which was a significant advance as it made a clear separation of geometry and topology in both its data structures and algorithms. This made it possible to implement generalised boolean operations and to systematically extend line geometric coverage and the fijunctionality of the modeller with operations such as blending. Subsequently other PhD theses supervised by Ian and based on the BUILD modellers included Dimensions and Tolerances (Hillyard 1978), Feature Recognition (Kyprianou 1980), Automatic 2D and 3D Mesh Generation (W^ldenweber 1982) and Surface Interseutions (Solomon 1985). These theses were some of the earliest



How to represent a curve?

- **There are three major types of object representation:**

(物体的表示方式主要有三种类型)

—Explicit representation (显示表示) : the explicit form of a curve in 2D gives the value of one variable, the dependent variable(因变量), in terms of the other, the independent variable(自变量). (二维空间中曲线的显示表示是以自变量来表达因变量的值) In x, y space, we may write

$$y = f(x)$$

—For the line, we usually write $y = mx + h$



-Implicit representation(隐式表示): In two dimensions, an implicit curve can be represented by the equation

$$f(x, y) = 0$$

-For the line

$$ax + by + c = 0$$

-For the circle

$$x^2 + y^2 - r^2 = 0$$

-Parametric form(参数形式): The parametric form of a curve expresses the value of each spatial variable for points on the curve in terms of an independent variable t , the parameter. (通过一个参数的形式来表达曲线点的每一个空间变量)

-In 3D, we have three explicit functions

$$x = x(t)$$

$$y = y(t)$$

$$z = z(t)$$

3D空间曲线上任意一个三维点 P 可表示为 $P(t)=[x(t), y(t), z(t)]$.

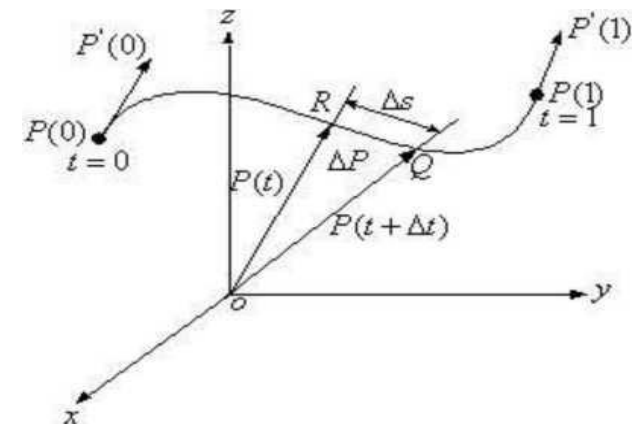
- 最简单的参数曲线是直线段，端点为 P_1 和 P_2 的直线段参数方程可表示为

$$P(t) = P_1 + (P_2 - P_1)t, \quad t \in [0, 1]$$

✓ 在几何造型系统中，**曲线曲面**方程通常表示成**参数形式**。



- One of the advantages of the parametric form is that it is the same in two and three dimensions. (它在二维和三维空间中具有相同的形式，容易推广到高维)
- A useful representation of the parametric form is to visualize the locus of points $P(t) = [X(t), Y(t), Z(t)]^T$ being drawn as t varies. (随着参数 t 的变化来实现点的轨迹的可视化，随着 t 从0到1慢慢变大，点的轨迹就是这条曲线的形状)





-We can think of the derivative(导数)
as the velocity with which the curve is traced out (将导数视为
曲线的绘制速度) and points in direction tangent to the curve
(该导数指向曲线的切向) .

$$\frac{dP(t)}{dt} = \begin{bmatrix} \frac{dx(t)}{dt} \\ \frac{dy(t)}{dt} \\ \frac{dz(t)}{dt} \end{bmatrix}$$

Parametric polynomial curves(参数多项式曲线)

- Parametric curves are not unique. A given curve or surface can be represented in many ways, but we shall find that parametric forms in which the functions are polynomials in t . (关心以 t 的多项式函数表达的参数形式)
- A polynomial parametric curve of degree 3 (3阶多项式参数曲线) is of form:

$$P(t) = at^3 + bt^2 + ct + d$$

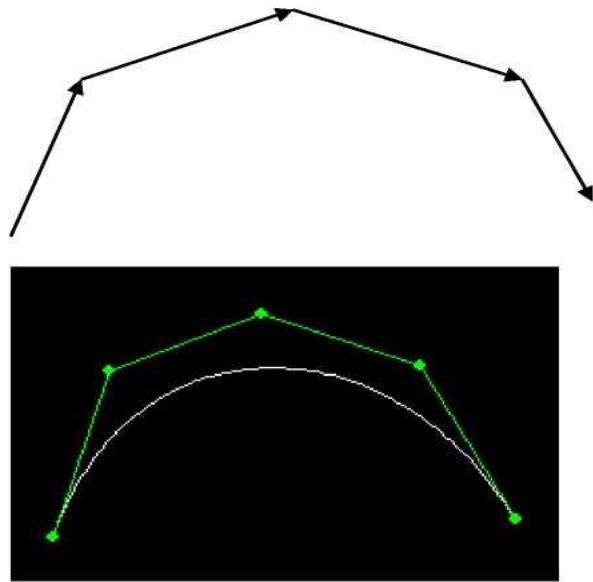
- It was known as **Ferguson curve**, and was used for **airplane design** in earlier in USA.



- But Ferguson curve is not straightforward, even the coefficients a , b , c , d are given, it's still hard to imagine the shape of the curve.
- 于是在1962年，Bezier开始了他的这项伟大工作，提出Bezier曲线。

Bezier Cuvres: Concept and properties

- Pierre Bezier proposed a method to represent a curve in terms of **connected vectors**.



$$V(t) = \sum_{i=0}^n f_{i,n}(t) A_i$$
$$f_{i,n}(t) = \begin{cases} 1, & i = 0, \\ \frac{(-t)^i}{(i-1)!} \frac{d^{i-1}}{dt^{i-1}} \left(\frac{(1-t)^{n-1} - 1}{t} \right) \end{cases}$$

- Curve can be designed interactively?
 - Curve are generated according to the control net (those connected vectors)
 - We may change vectors to modify the curve. (exe示例)



- However, such a definition is unintelligible(难以理解的)
- 1972, Forrest published his famous paper in Computer Aided Design journal.
 - he pointed out that, **Bezier curve** can be defined in terms of points with help of Bernstein Polynomials.
- 在Forrest之后，我们来看基于Bernstein Polynomials如何定义Bezier curve。



Definition

- Given control points $P_0, P_1, \dots, P_n$, Bezier curve can be defined as ($P(t)$ 表示的是曲线上任一点的位置矢量，表示的是 **n 次Bezier曲线**)

$$P(t) = \sum_{i=0}^n P_i B_{i,n}(t), \quad t \in [0,1]$$

其中： $B_{i,n}(t)$ 是第 i 个 n 阶 *Bernstein* 多项式
(也称为 **n 次Bernstein基函数**)

$$B_{i,n}(t) = C_n^i t^i (1-t)^{n-i} = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i}$$

$(i = 0, 1, \dots, n)$

注意：当 $i=0$ ， $t=0$ 时， $t^i=1$ ， $i!=1$ 。

□ 一次Bezier曲线 (n=1) 恰好是连接起点p 0 和终点p 1 的直线段!

当n=1时, 有两个控制点 p_0 和 p_1 , Bezier多项式是一次多项式:

$$p(t) = \sum_{i=0}^1 P_i B_{i,1}(t) = (1-t)P_0 + tP_1 \quad t \in [0, 1]$$

□ 二次Bezier曲线 (n=2) 二次Bezier曲线为抛物线

Bezier多项式是二次多项式:

$$\begin{aligned} p(t) &= \sum_{i=0}^2 P_i B_{i,2}(t) = (1-t)^2 P_0 + 2t(1-t)P_1 + t^2 P_2 \\ &= (P_2 - 2P_1 + P_0)t^2 + 2(P_1 - P_0)t + P_0 \end{aligned} \quad t \in [0, 1]$$



□ 三次Bezier曲线 (n=3)

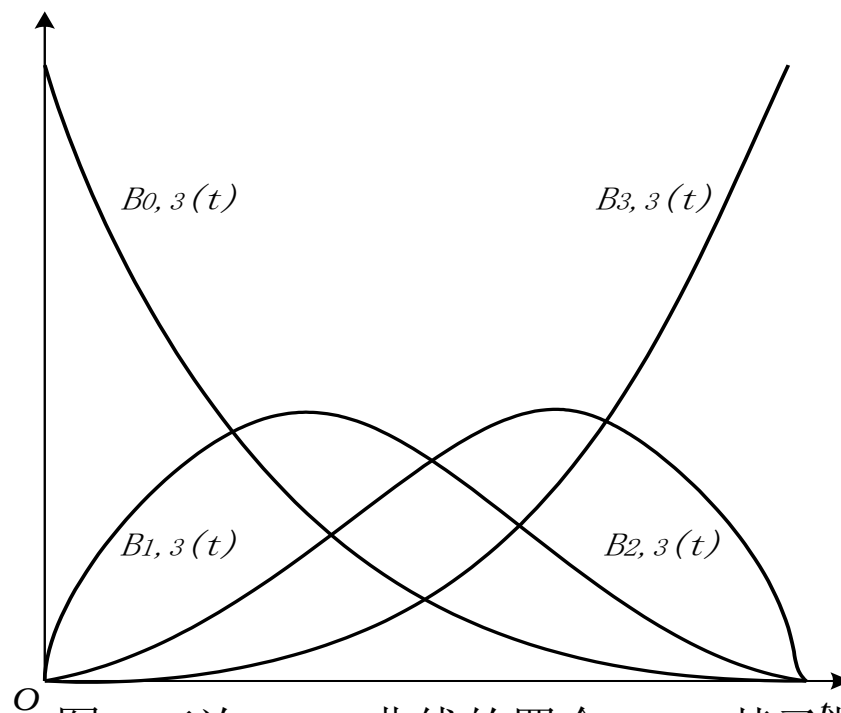
Bezier多项式是**三次多项式**:

$$p(t) = \sum_{i=0}^3 P_i B_{i,3}(t)$$

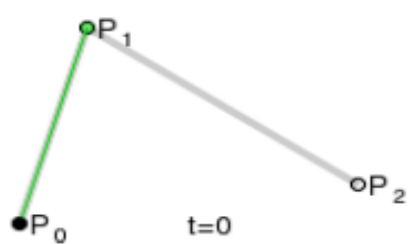
$$= B_{0,3}(t)P_0 + B_{1,3}(t)P_1 + B_{2,3}(t)P_2 + B_{3,3}(t)P_3$$

$$= (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t)P_2 + t^3 P_3$$

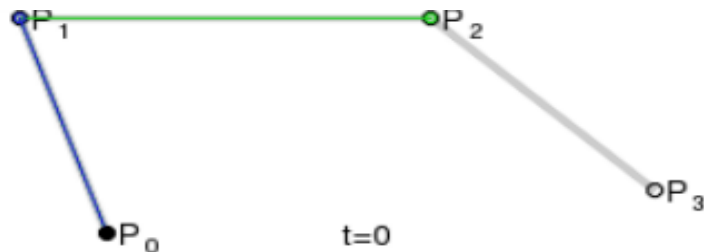
$$t \in [0, 1]$$



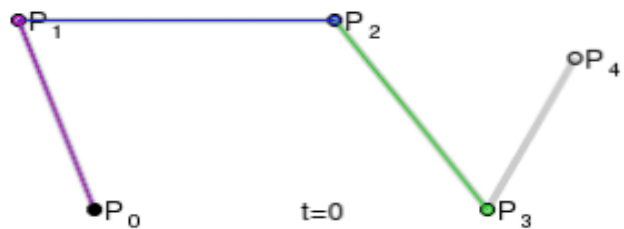
图：三次Bezier曲线的四个Bezier基函数



二阶



三阶



四阶

三个 Bézier 曲线

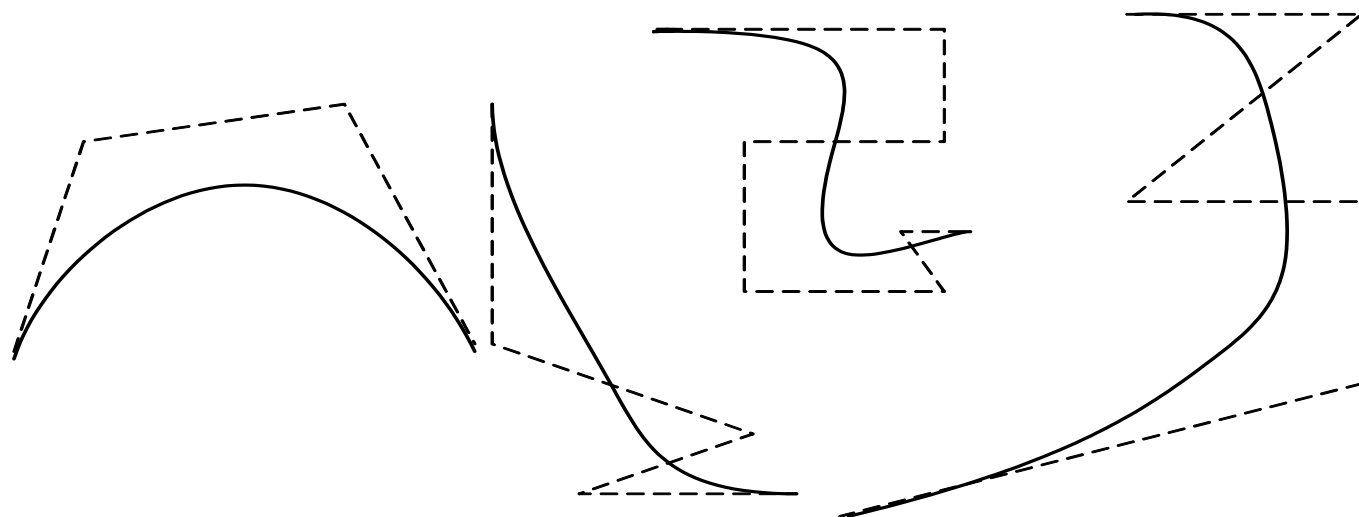


图: Bezier曲线

Property of *Bernstein polynomial*



- 非负 (Non-negative)

$$B_{i,n}(t) = \begin{cases} = 0 & t = 0, 1 \\ > 0 & t \in (0, 1), i = 1, 2, \dots, n-1; \end{cases}$$

- 端点 (End point)

$$B_{i,n}(0) = \begin{cases} 1 & (i = 0) \\ 0 & \text{otherswise} \end{cases}$$

$$B_{i,n}(1) = \begin{cases} 1 & (i = n) \\ 0 & \text{otherswise} \end{cases}$$



- 归一性 (Unity) 基函数求和等于1

$$\sum_{i=0}^n B_{i,n}(t) \equiv 1 \quad t \in (0,1)$$

— 证明：根据二项式定理 (*Binomial Theorem*), 可以得到

$$\sum_{i=0}^n B_{i,n}(t) = \sum_{i=0}^n C_n^i t^i (1-t)^{n-i} = [(1-t) + t]^n \equiv 1$$



- 对称性 (Symmetry) 第*i*个基函数与倒数第*i*个基函数是对称的

$$B_{i,n}(t) = B_{n-i,n}(1-t), \quad i = 0, 1, \dots, n$$

— 证明：

由伯恩斯坦基函数定义：

$$B_{i,n}(t) = C(n, i) t^i (1-t)^{n-i},$$

其中 $C(n, i) = \frac{n!}{i!(n-i)!}$ 。

计算右边的 $B_{n-i,n}(1-t)$ ：

$$B_{n-i,n}(1-t) = C(n, n-i) (1-t)^{n-i} (1-(1-t))^i = C(n, n-i) (1-t)^{n-i} t^i.$$

利用组合数的对称性质 $C(n, n-i) = C(n, i)$ ，代入得：

$$B_{n-i,n}(1-t) = C(n, i) t^i (1-t)^{n-i} = B_{i,n}(t).$$



- 递归性 (Recursive)

$$B_{i,n}(t) = (1-t)B_{i,n-1}(t) + tB_{i-1,n-1}(t),$$
$$(i = 0, 1, \dots, n)$$

– 这个式子表明 n 阶 *Bernstein* 多项式是 $n-1$ 阶 *Bernstein* 多项式的线性组合。

$$\begin{aligned} B_{i,n}(t) &= C_n^i t^i (1-t)^{n-i} = (C_{n-1}^i + C_{n-1}^{i-1}) t^i (1-t)^{n-i} \\ &= (1-t) C_{n-1}^i t^i (1-t)^{(n-1)-i} + t C_{n-1}^{i-1} t^{i-1} (1-t)^{(n-1)-(i-1)} \\ &= (1-t) B_{i,n-1}(t) + t B_{i-1,n-1}(t) \end{aligned}$$



(经常要求导，比如计算曲线的切向量。
伯恩斯坦多项式的导数正好是两个伯恩斯坦多项式的差)

- 导数 (Derivation)

$$B'_{i,n}(t) = n[B_{i-1,n-1}(t) - B_{i,n-1}(t)],$$
$$i = 0, 1, \dots, n;$$

- 最大值

- $B_{i,n}(t)$ 在 $[0,1]$ 之间的 $t = i/n$ 处有唯一的局部极大值。



- 升阶 (degree raising) 公式

$$(1-t)B_{i,n}(t) = \left(1 - \frac{i}{n+1}\right)B_{i,n+1}(t)$$

$$tB_{i,n}(t) = \frac{i+1}{n+1}B_{i+1,n+1}(t)$$

$$B_{i,n}(t) = \left(1 - \frac{i}{n+1}\right)B_{i,n+1}(t) + \frac{i+1}{n+1}B_{i+1,n+1}(t)$$

- 积分 (Integral)

$$\int_0^1 B_{i,n}(t) dt = \frac{1}{n+1}$$

Property of Bezier curve



- End point properties(端点性质)

- Position of end point

- According to the end position's property of *Bernstein polynomial*, We have

$$P(0)=P_0, \quad P(1) = P_n ,$$

- So the start point and end point of Bezier curve coincide with start point and end point of the control polygon.



-Tangent Vector (切向量)

- Since

$$P'(t) = n \sum_{i=0}^{n-1} P_i [B_{i-1,n-1}(t) - B_{i,n-1}(t)]$$

- We have

$$P'(0) = n(P_1 - P_0), \quad P'(1) = n(P_n - P_{n-1})$$

- This implies the tangent vector of the curve at the start and end points is the same with the direction of the first and last edges of the control polygon.

(起点和终点处曲线的切向量方向与控制多边形第一条及最后一条边的方向一致)



- 二阶导数 (Second Derivative)

$$P''(t) = n(n-1) \sum_{i=0}^{n-2} (P_{i+2} - 2P_{i+1} + P_i) B_{i,n-2}(t)$$

因此

该式表明二阶导只与相邻的三个顶点有关。

$$P''(0) = n(n-1) (P_2 - 2P_1 + P_0)$$

$$P''(1) = n(n-1) (P_n - 2P_{n-1} + P_{n-2})$$

由曲率公式可得

$$k(0) = \frac{n-1}{n} \square \frac{|(P_1 - P_0) \square (P_2 - P_1)|}{|P_1 - P_0|^3}$$

$$k(1) = \frac{n-1}{n} \square \frac{|(P_{n-1} - P_{n-2}) \square (P_n - P_{n-1})|}{|P_n - P_{n-1}|^3}$$

上式表示Bezier曲线在端点处的曲率（曲率是微分几何中刻画曲线弯曲程度的一个概念）



- Difference form of k-th Derivative (k 阶导数的差分表示)
(n 次 Bezier 曲线的 k 阶导数可用如下差分公式表示)

$$P^k(t) = \frac{n!}{(n-k)!} \sum_{i=0}^{n-k} \Delta^k P_i B_{i,n-k}(t) \quad t \in [0,1]$$

k 阶差分

- The high-order forward difference vector is recursively defined by low-order forward difference vector (高阶前向差分向量可以通过低阶前向差分向量递归定义)

$$\Delta^0 P_i = P_i \quad (0 \text{ 阶差分是自己})$$

$$\Delta^k P_i = \Delta^{k-1} P_{i+1} - \Delta^{k-1} P_i \quad (k \text{ 阶差分是 } k-1 \text{ 阶差分的差})$$



- Symmetry (对称)

– 控制顶点为 $P_i^* = P_{n-i}, (i = 0, 1, \dots, n)$ 的曲线保留了曲线 $P(t)$ 的形状，但方向与之相反。

$$\begin{aligned} C^*(t) &= \sum_{i=0}^n P_i^* B_{i,n}(t) = \sum_{i=0}^n P_{n-i} B_{i,n}(t) = \sum_{i=0}^n P_{n-i} B_{n-i,n}(1-t) \\ &= \sum_{i=0}^n P_i B_{i,n}(1-t), \quad t \in [0, 1] \end{aligned}$$



• Convex hull Property (凸包性质)

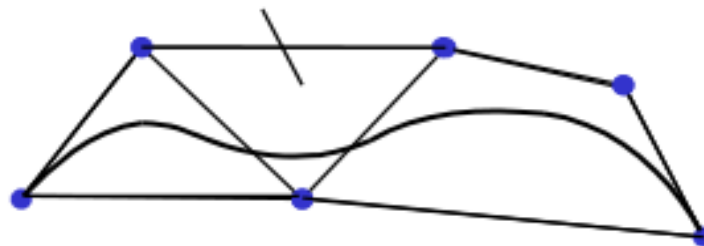
(给定空间上的 n 个点, 凸包就是包含这 n 个点的最小的凸多边形)

— 根据

$$\sum_{i=0}^n B_{i,n}(t) \equiv 1 \quad (\text{归一性})$$

$$0 \leq B_{i,n}(t) \leq 1 (0 \leq t \leq 1, i = 0, 1, \dots, n)$$

— 曲线 $P(t)$ 在所有控制点的凸包内部



(**Bezier**曲线 $P(t)$ 在 $t \in [0, 1]$ 中各点是控制点 P_i 的线性组合, 即曲线落在 P_i 构成的凸包之中)



- Geometric invariance (几何不变性)

- It means that some geometry property doesn't change with Coordinates varying.(一些几何性质不会随着坐标系的变化而改变)

- The position and shape of Bezier curve are dependent on vertex P_i of control polygon, but not the Coordinates.

- (Bézier 曲线的位置和形状依赖于控制多边形顶点 P_i 的位置，而不是坐标系)

Bezier曲线的生成



生成一条Bezier曲线实际上就是要求出曲线上的点。下面介绍两种曲线生成的方法：

1、根据定义（Bezier曲线方程）直接生成Bezier曲线

绘制Bezier曲线主要有以下步骤：

$$p(t) = \sum_{i=0}^n P_i B_{i,n}(t) \quad t \in [0,1]$$

$$B_{i,n}(t) = \frac{n!}{i!(n-i)!} t^i (1-t)^{n-i} = C_n^i t^i (1-t)^{n-i} \quad (i = 0, 1, \dots, n)$$

① 首先给出 C_n^i 的递归计算式：

$$C_n^i = \frac{n!}{i!(n-i)!} = \frac{n-i+1}{i} C_n^{i-1} \quad n \geq i$$

② 将 $p(t) = \sum_{i=0}^n P_i B_{i,n}(t)$ 表示成分量坐标形式：

$$x(t) = \sum_{i=0}^n x_i B_{i,n}(t)$$

$$y(t) = \sum_{i=0}^n y_i B_{i,n}(t) \quad t \in [0,1]$$

$$z(t) = \sum_{i=0}^n z_i B_{i,n}(t)$$

根据以上的公式可以直接写出绘制Bezier曲线的程序

$$C_n^i = \frac{n!}{i!(n-i)!} = \frac{n-i+1}{i} C_n^{i-1} \quad n \geq i$$

P_i 代表空间的很多点， t 在0到1之间，把 t 代进去可以算出一个数——即平面或空间一个点

随着 t 值的变化，点也在变化。当 t 从0变到1时，就得到空间的一个图形，这个图形就是bezier曲线

举例：



给定型值点 $(0,0),(0,100),(100,0),(100,100)$ ，如对应的参数为 $0, \frac{1}{3}, \frac{2}{3}, 1$ ，反求插值这四个型值点的三次 Bezier 曲线的控制点。

对三次 Bezier 曲线， $C(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t) P_2 + t^3 P_3$ ，

假设控制顶点为 b_0, b_1, b_2, b_3

$$b_0 = (0,0),$$

$$b_3 = (100,100),$$

$$(0,100) = \frac{8}{27}b_0 + \frac{4}{9}b_1 + \frac{2}{9}b_2 + \frac{1}{27}b_3$$

$$(100,0) = \frac{1}{27}b_0 + \frac{2}{9}b_1 + \frac{4}{9}b_2 + \frac{8}{27}b_3$$

解方程组可得 $b_1 = (-\frac{350}{3}, \frac{1000}{3})$, $b_2 = (\frac{650}{3}, -\frac{700}{3})$ 。



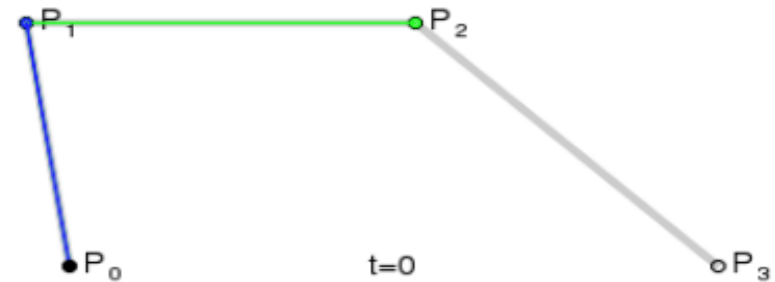
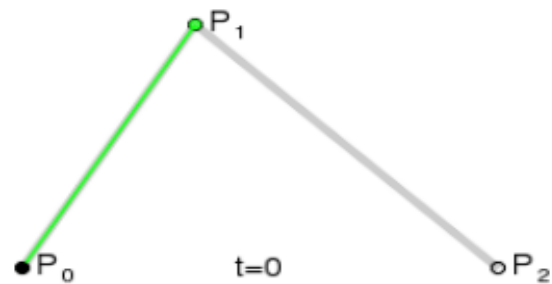
2、de Casteljau Algorithm (递推de Casteljau算法)

- In industry application, it's required to evaluate a point on the curve at parameter t . (曲线上参数 t 位置的点)
- We didn't evaluate the value by the Bezier Curve Equation, but use a recursive algorithm, which is numerical stable.

根据Bezier曲线的定义确定的参数方程绘制Bezier曲线,因其计算量过大,不太适合在工程上使用。

de Casteljau提出的递推算法则要简单得多。

de Casteljau递推算法的过程一定比分割

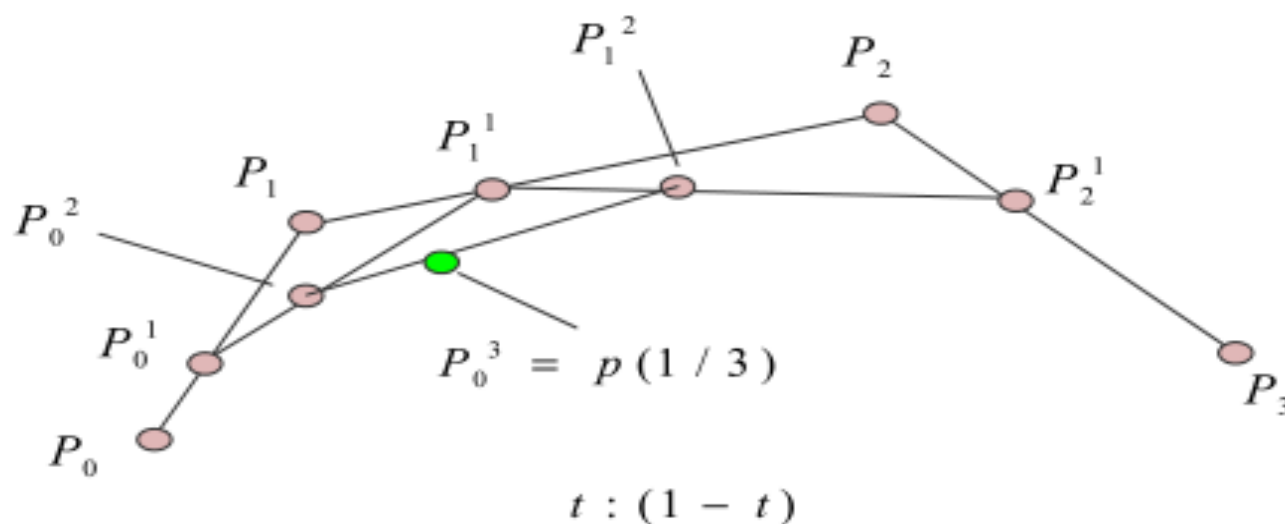


Bezier曲线上的任一个点(t)，都是其它相邻线段的同等比例(t)点处的连线，再取同等比例(t)的点再连线，一直取到最后那条线段的同等比例(t)处，该点就是Bezier曲线上的点(t)

de Casteljau算法几何作图



(3) 重复进行下去，直到n级递推得到一个中间顶点 $P(t)$ ，即为所求曲线上的点

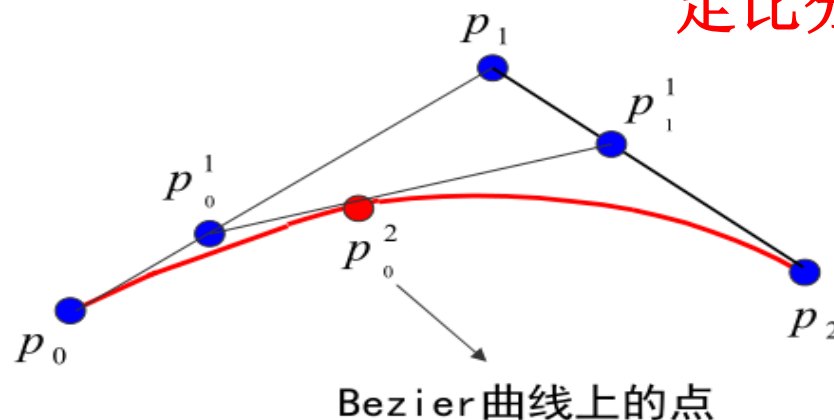


de Casteljau递推公式的分析



以二次Bezier曲线为例，求曲线上 $t=1/3$ 的点：

$$\begin{aligned}P_0^1 &= (1-t)P_0 + tP_1 \\P_1^1 &= (1-t)P_1 + tP_2 \\P_0^2 &= (1-t)P_0^1 + tP_1^1\end{aligned}$$



t 从0变到1，第一、二式就分别表示控制二边形的第一、二条边，它们是两条一次Bezier曲线。将一、二式代入第三式得：

$$P_0^2 = (1-t)^2 P_0 + 2t(1-t)P_1 + t^2 P_2$$

当 t 从0变到1时，它表示了由三顶点 P_0 、 P_1 、 P_2 三点定义的一条二次Bezier曲线

二次Bezier曲线 P_0^2 可以定义为分别由前两个顶点(P_0, P_1)和后两个顶点(P_1, P_2)决定的一次Bezier曲线的线性组合

由 $(n+1)$ 个控制点 $P_i (i=0, 1, \dots, n)$ 定义的 n 次Bezier曲线 P_0^n 可被定义为分别由前、后 n 个控制点定义的两条 $(n-1)$ 次Bezier曲线 P_0^{n-1} 与 P_1^{n-1} 的线性组合：

$$P_0^n = (1 - t) P_0^{n-1} + t P_1^{n-1} \quad t \in [0, 1]$$

由此得到Bezier曲线的递推计算公式：

$$P_i^k = \begin{cases} P_i & k = 0 \\ (1 - t) P_i^{k-1} + t P_{i+1}^{k-1} & k = 1, 2, \dots, n, i = 0, 1, \dots, n - k \end{cases}$$

第 k 级中间顶点：表示第 k 级递推得到的中间顶点（即定比分割点）。

然后对这些中间顶点构成的控制多边形再执行同样的定比分割，得到下一级中间顶点。

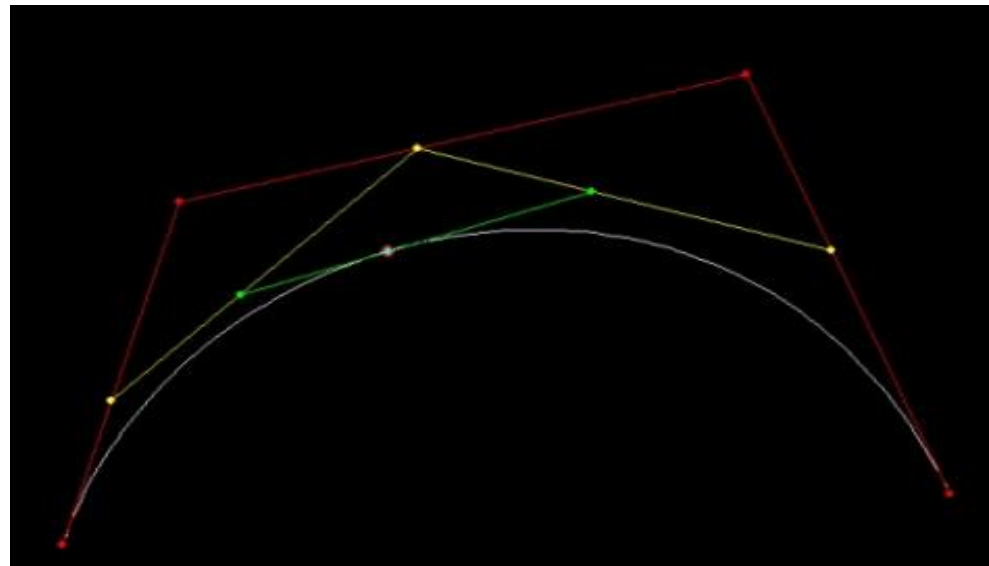
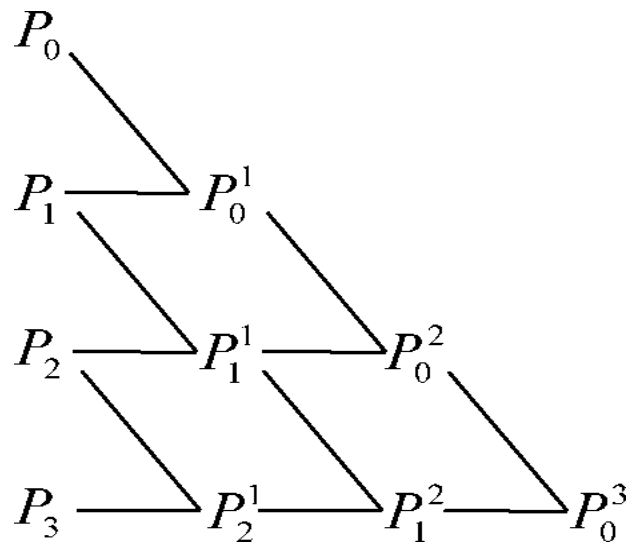
这便是著名的de Casteljau算法。用这一递推公式，在给定参数下，求Bezier曲线上一点 $P(t)$ 非常有效

de Casteljau算法稳定可靠，直观简便，可以编出十分简捷的程序，是计算Bezier曲线的基本算法和标准算法。

- When $n = 3$, the recursive procedure is illustrated by the following figure(de Casteljau算法递推出的 p_i^k 呈直角三角形)

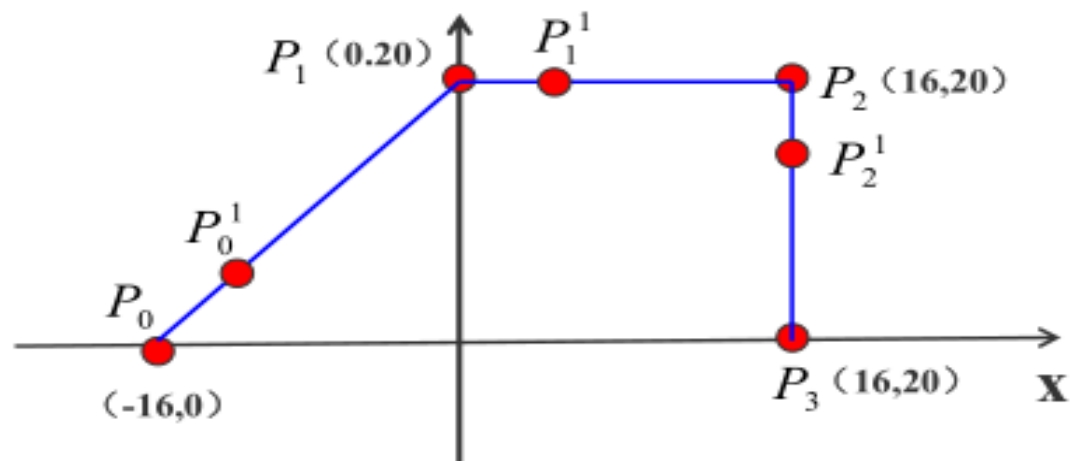
由此得到Bezier曲线的递推计算公式：

$$P_i^k = \begin{cases} P_i & k = 0 \\ (1-t)P_i^{k-1} + tP_{i+1}^{k-1} & k = 1, 2, \dots, n, i = 0, 1, \dots, n-k \end{cases}$$

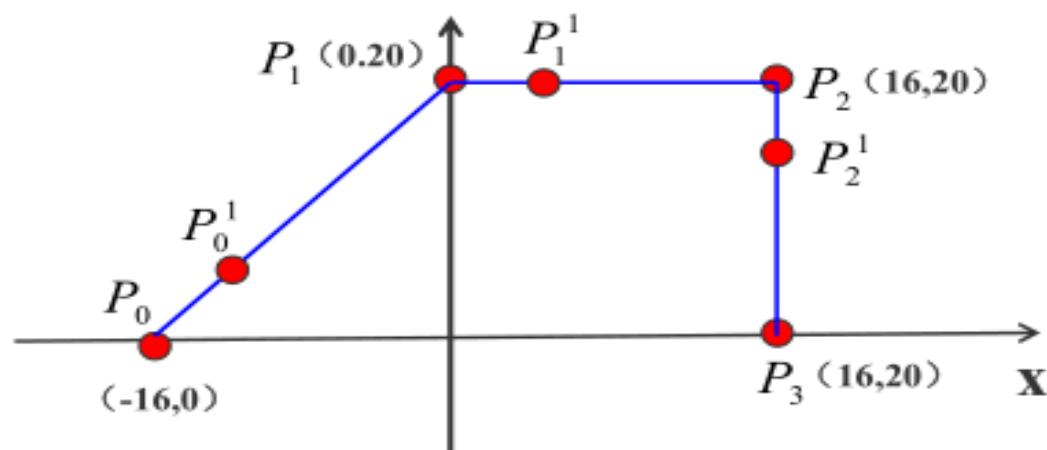


Bezier曲线计算举例

计算参数为 $t=1/4$ 的P值
并画出 de Casteljau 三角形



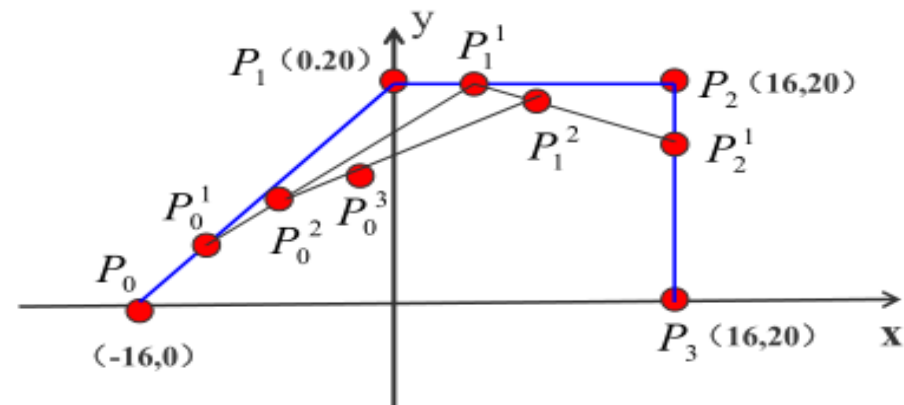
$$P_i^k = \begin{cases} P_i & k = 0 \\ (1-t)P_i^{k-1} + tP_{i+1}^{k-1} & k = 1, 2, \dots, n, i = 0, 1, \dots, n-k \end{cases}$$



$$P_0^1 = (1-t)P_0 + tP_1 = (1-\frac{1}{4})P_0 + \frac{1}{4}P_1 = \frac{3}{4}[-16,0] + \frac{1}{4}[0,20] = [-12,5]$$

$$P_1^1 = (1-t)P_1 + tP_2 = (1-\frac{1}{4})P_1 + \frac{1}{4}P_2 = \frac{3}{4}[0,20] + \frac{1}{4}[16,20] = [4,20]$$

$$P_2^1 = (1-t)P_2 + tP_3 = (1-\frac{1}{4})P_2 + \frac{1}{4}P_3 = \frac{3}{4}[16,20] + \frac{1}{4}[16,0] = [16,15]$$



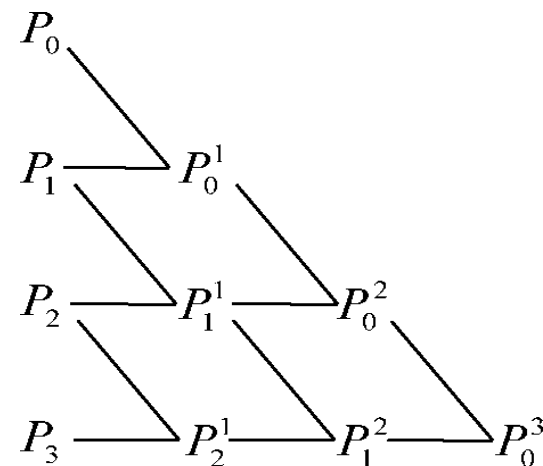
$$P_0^2 = (1-t)P_0^1 + tP_1^1 = (1-\frac{1}{4})P_0^1 + \frac{1}{4}P_1^1 = \frac{3}{4}[-12,5] + \frac{1}{4}[4,20] = [-8,8.75]$$

$$P_1^2 = (1-t)P_1^1 + tP_2^1 = (1-\frac{1}{4})P_1^1 + \frac{1}{4}P_2^1 = \frac{3}{4}[-12,5] + \frac{1}{4}[4,20] = [-8,8.75]$$

$$P_0^3 = (1-t)P_0^2 + tP_1^2 = (1-\frac{1}{4})P_0^2 + \frac{1}{4}P_1^2 = \frac{3}{4}[-8,8.75] + \frac{1}{4}[7,18.75]$$

$$= [-4.25, 11.25] = P(\frac{1}{4})$$

de Casteljau 三角形



de Casteljau算法应用举例2:

设一条三次 Bezier 曲线的前三个控制顶点为 $(30,0)$, $(60,20)$, $(80,20)$, 曲线在 $t = \frac{1}{2}$ 处的值为 $(70, 15)$, 试求最后一个控制顶点。

解: (1) 由 de Casteljau 算法 $P_i^r = (1-t)P_i^{r-1} + tP_{i+1}^{r-1}$

$t=1/2$, 可求得: $P_0^1 = (45,10)$, $P_1^1 = (70,20)$, $P_2^1 = (57.5,15)$

同理可反推角点: $P_0^3 = 1/2(P_0^2 + P_1^2) = (70,15)$, 可得 $P_1^2 = (82.5,15)$

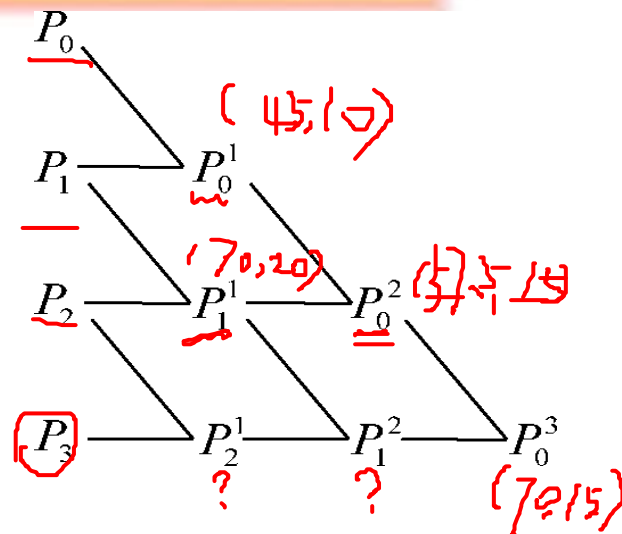
类似可得, $P_2^1 = (95,10)$, $P_3 = (110,0)$

(2) 对三次 Bezier 曲线, $C(t) = (1-t)^3 P_0 + 3t(1-t)^2 P_1 + 3t^2(1-t) P_2 + t^3 P_3$, 设

$P_3(x_3, y_3)$, 则有

$$\begin{cases} (1-\frac{1}{2})^3 \times 30 + 3 \times \frac{1}{2} \times (1-\frac{1}{2})^2 \times 60 + 3 \times (\frac{1}{2})^2 \times (1-\frac{1}{2}) \times 80 + (\frac{1}{2})^3 \times x_3 = 70 \\ (1-\frac{1}{2})^3 \times 0 + 3 \times \frac{1}{2} \times (1-\frac{1}{2})^2 \times 20 + 3 \times (\frac{1}{2})^2 \times (1-\frac{1}{2}) \times 20 + (\frac{1}{2})^3 \times y_3 = 15 \end{cases}$$

解得 $x_3 = 110, y_3 = 0$, 即最后一个控制顶点为 $(110,0)$ 。



Geometric continuity(几何连续性)

用于Bezier曲线的拼接



几何设计中，一条Bezier曲线往往难以描述复杂的曲线形状。这是由于增加特征多边形的顶点数，会引起Bezier曲线次数的提高，而高次多项式又会带来计算上的困难

$$p(t) = \sum_{i=0}^n P_i B_{i,n}(t) \quad t \in [0,1]$$

$$B_{i,n}(t) = C_n^i t^i (1-t)^{n-i} \quad (i = 0, 1, \dots, n)$$

- In CAD applications, it's not encouraged to design a curve by high degree Bezier curve（高阶Bézier曲线）.
- It's common to use lower degree Bezier curves with smooth connections（光滑链接的低阶 Bézier 曲线）.
- Can we use traditional concept of continuity（连续性）？

曲线连续性



当许多参数曲线段首尾相连构成一条曲线时，如何保证各曲线段在连接处具有合乎要求的连续性是一个重要问题。

假定参数曲线段 p_i 以参数形式进行描述：

$$p_i = p_i(t) \quad t \in [t_{i0}, t_{i1}]$$

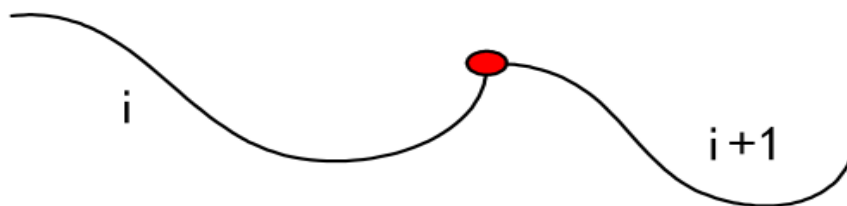
介绍参数曲线两种意义上的连续性：即参数连续性和几何连续性

(1) 参数连续性

0阶参数连续性:

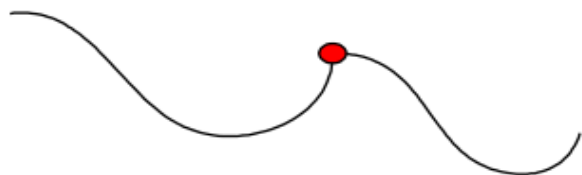
记作 C^0 连续性, 是指曲线的几何位置连接, 即第一个曲线段在 t_{i1} 处的 x, y, z 值与第二个曲线段在 $t_{(i+1)0}$ 处的 x, y, z 值相等:

$$p_i(t_{i1}) = p_{(i+1)}(t_{(i+1)0})$$



2阶参数连续性:

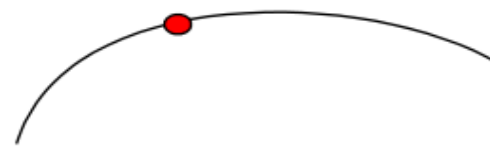
记作 C^2 连续性, 指两个相邻曲线段的方程在相交点处具有相同的一阶和二阶导数。类似地, 还可定义高阶参数连续性



(a) 0阶连续性



(b) 1阶连续性



(c) 2阶连续性

◆ 对于 C^2 连续性, 交点处的切向量变化率相等, 即切线从一个曲线段平滑地变化到另一个曲线段, 二阶连续性对电影中的动画途径和很多精密CAD需求有用。

参数连续性的disadvantage

经典的参数连续性在图形学里是不适合的，因为太苛刻，所以引进了几何连续性的概念

汽车曲面的设计美观要求很高，但有时候车身的一条曲线并不是参数连连续的，但人眼看上去已经是很光滑的了，因此需要一种更弱的连续性

$$\Phi(t) = \begin{cases} V_0 + \frac{V_1 - V_0}{3}t, & 0 \leq t \leq 1 \\ V_0 + \frac{V_1 - V_0}{3} + (t-1)\frac{2(V_1 - V_0)}{3}, & 1 \leq t \leq 2 \end{cases}$$
$$\Phi'(1^-) = \frac{1}{3}(V_1 - V_0) \quad \Phi'(1^+) = \frac{2}{3}(V_1 - V_0)$$

- ◆ 上面的函数实际上是 V_0 V_1 的直线方程。但可以发现，在 $t=1$ 这一点，直线的左右导数不相等！
- ◆ 这意味着该函数不是连续的。但该方程事实上是一条直线。
- ◆ 这种现象意味着关于连续性的传统观念并不适合用于描述CAD和图形学中形状的连续性。我们采用“几何连续性”

几何连续性



曲线段相连的另一个连续性条件是几何连续性。与参数连续性不同的是，它只需曲线段在相交处的参数导数成比例即可

0阶几何连续性：记作 G^0 连续性。与0阶参数连续性的定义相同，满足：

$$p_i(t_{i1}) = p_{(i+1)}(t_{(i+1)0}) \quad \text{几何位置相接}$$

1阶几何连续性，记作 G^1 连续性。若要求在结合处达到 G^1 连续，就是说两条曲线在结合处在满足 G^0 连续的前提下，并有公共的切矢

$$Q'(0) = \alpha P'(1) \quad (\alpha > 0)$$

2阶几何连续性，记作 G^2 连续性。就是说两条曲线在结合处在满足 G^1 连续的前提下，并有公共的曲率

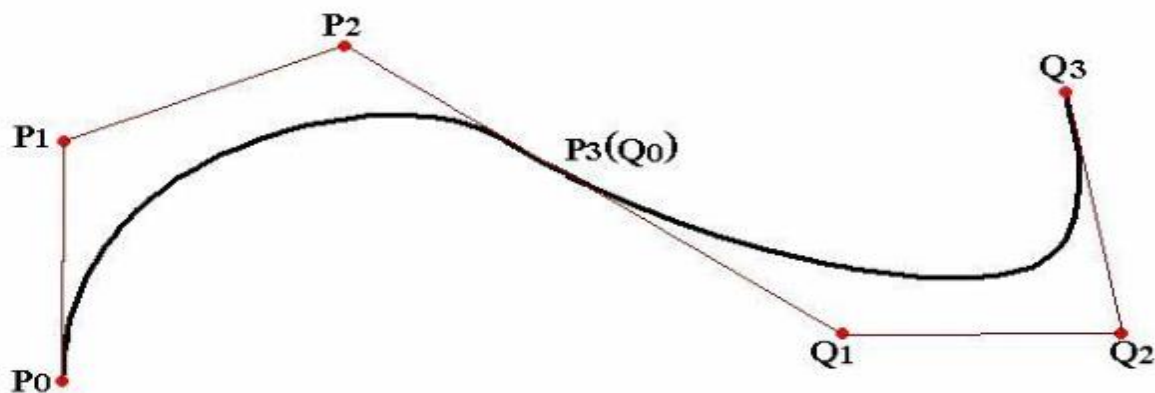
- ◆ 一阶导数相等和有公共切向量这两个概念差别是什么？导数相等是大小方向都相等，而公共切矢意味着方向相同但大小不等。
- ◆ 如果是几何连续的话，只是要求切向量一样，方向一样，长度可以不同。条件减弱了。

Bezier曲线的拼接



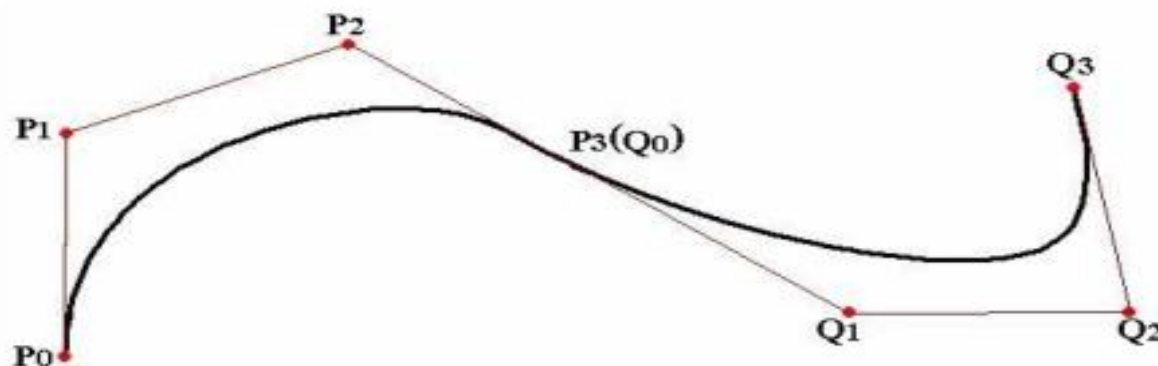
- 由于增加特征多边形的顶点数，会引起Bezier曲线次数的提高，而高次多项式又会带来计算上的困难。
- 采用分段设计，然后将各段曲线相互连接起来，并在接合处保持一定的连续条件。（这里考虑几何连续性）

给定两条Bezier曲线 $P(t)$ 和 $Q(t)$ ，相应控制点为 $P_i (i=0, 1, \dots, n)$ 和 $Q_i (i=0, 1, \dots, m)$





(1) 要使它们达到
 G^0 连续, 则: $P_n = Q_0$



(2) 要使它们达到 G^1 连续, 只要保证 P_{n-1} , $P_n=Q$, Q_1 三点共线就行了 G^1 连续 (切向连续)

Bezier曲线的起点和终点处的切线方向和特征多边形的第一条边及最后一条边的走向一致

□ 在两段三次Bezier曲线间得到 G^2 连续性, G^2 连续 (曲率连续)

$$p_2''(0) = \beta \cdot p_1''(1)$$

$$(Q_0 - 2Q_1 + Q_2) = \beta \cdot (P_1 - 2P_2 + P_3)$$



Degree raising/elevation (升阶)

- Degree raising means that adding control points to raise the degree of the **Bezier curves**, but **the shape and direction of the curve remain unchanged**.
 - Degree raising increases the flexibility of shape control.
 - After degree raising, control points are changed.
 - How to raising the degree of a polynomial? **Bezier curves?**
(怎样增加多项式的阶? 怎样增加 Bézier 曲线的阶?)

- Please note

$$\begin{aligned} P(t) &= \sum_{i=0}^n P_i B_{i,n}(t) = \sum_{i=0}^n P_i ((1-t) + t) B_{i,n}(t) \\ &= \sum_{i=0}^{n+1} \left(\frac{n+1-i}{n+1} P_i + \frac{i}{n+1} P_{i-1} \right) B_{i,n+1}(t) \end{aligned}$$



- 我们可以得到升阶公式:

$$P_i^* = \frac{i}{n+1} P_{i-1} + \left(1 - \frac{i}{n+1} \right) P_i \quad (i = 0, 1, \dots, n+1)$$

- 上述公式说明

- 新的控制顶点是旧的控制顶点的系数为 $i/(n+1)$ 的线性组合。
- 新的控制多边形在旧控制多边形的凸包内部。
- 新的控制多边形更加靠近曲线。

- 计算举例

给定三次 Beizer 曲线的控制顶点 $(0,0), (0,100), (100,0), (100,100)$ ，计算升阶一次后的控制顶点。 升阶一次后的控制顶点为 $(0,0), (0,75), (50,50), (100,25), (100,100)$



Degree reduction (降阶)

- Degree reduction is the opposite of degree raising (逆过程)
 - Degree reduction is to find a curve defined by new control points with minimum error. (降阶的目的是寻找一组新的控制点定义曲线来逼近原始曲线，使得误差最小)

$$P_i^* (i = 0, 1, \dots, n-1)$$

-Suppose P_i is result of degree raising P_i^* : $P_i^* (i = 0, 1, \dots, n-1)$

假设 P_i 是 P_i^* 升阶的结果:

$$P_i = \frac{n-i}{n} P_i^* + \frac{i}{n} P_{i-1}^*$$

- 我们可以得到两个递推公式
- 第一个递推式在靠近 P_0 处趋向生成较好的逼近
第二个递推式在靠近 P_n 处趋向生成较好的逼近

$$P_i^\# = \frac{nP_i - iP_{i-1}^\#}{n-i} \quad i = 0, 1, \dots, n-1$$

$$P_{i-1}^* = \frac{nP_i - (n-i)P_i^\#}{i} \quad i = n, n-1, \dots, 1$$

- 因此我们有两种降阶方案 综合利用上面两个式子进行降阶, 但仍然不精确

– Forrest (1972)

$$\hat{P}_i = \begin{cases} P_i^\#, & i = 0, 1, \dots, \left\lfloor \frac{n-1}{2} \right\rfloor \\ P_i^*, & i = \left\lfloor \frac{n-1}{2} \right\rfloor + 1, \dots, n-1 \end{cases}$$

– Farin (1983)

$$\hat{P}_i = \left(1 - \frac{i}{n-1}\right) P_i^\# + \frac{i}{n-1} P_i^*$$

Bezier surface

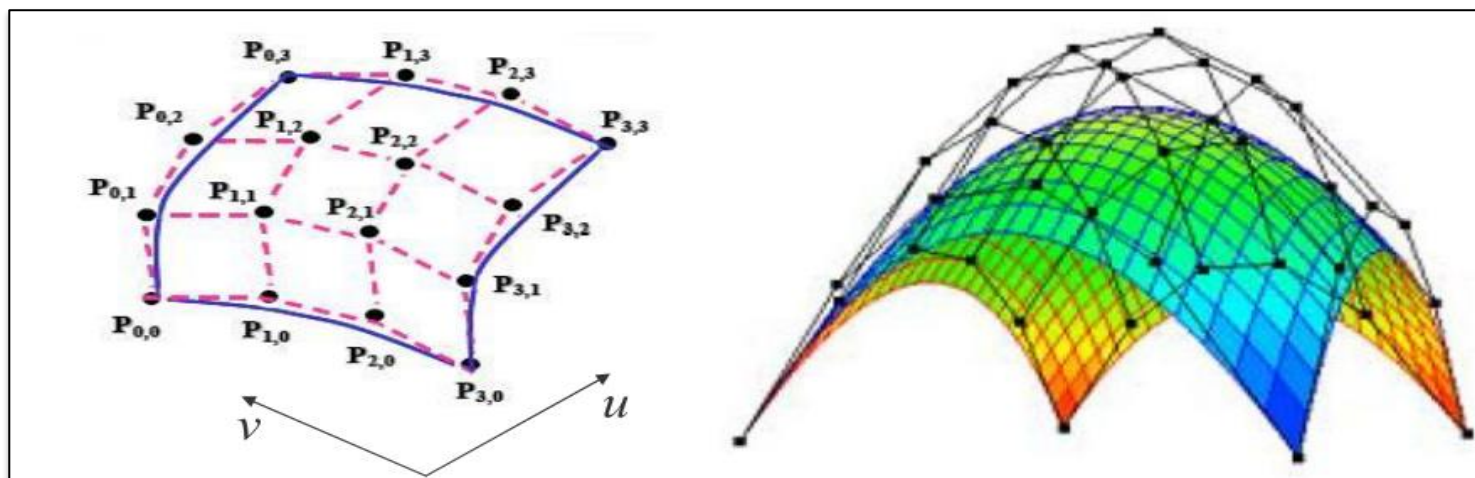


- Bezier Surface (Rectangular (矩形域))

- 假设 $P_{ij}(0,1,\dots,m; j=0,1,\dots,n)$ 是 $(m+1) \times (n+1)$ 的控制点集, $m \times n$ 阶矩形域 Bézier 曲面可以通过张量积的形式定义。 ($m \times n$ 次 Bezier 曲面)

$$P(u, v) = \sum_{i=0}^m \sum_{j=0}^n P_{ij} B_{i,m}(u) B_{j,n}(v) \quad u, v \in [0,1]$$

- 其中 $B_{i,m}(u) = C_m^i u^i (1-u)^{m-i}$ 和 $B_{j,n}(v) = C_n^j v^j (1-v)^{n-j}$ 是 *Bernstein* 多项式。



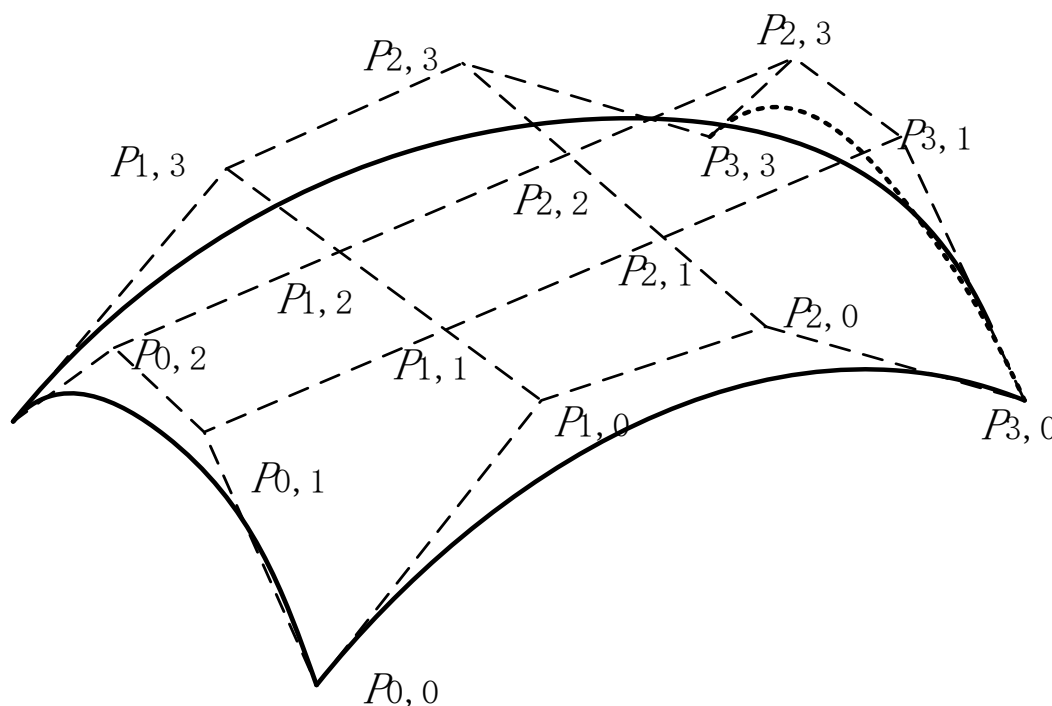
依次用线段连接点列中相邻两点所形成的空间网格, 称之为特征网格

形成一个**双三次Bezier曲面片**，其控制网格的控制点数应为多少？
答：16个控制点。

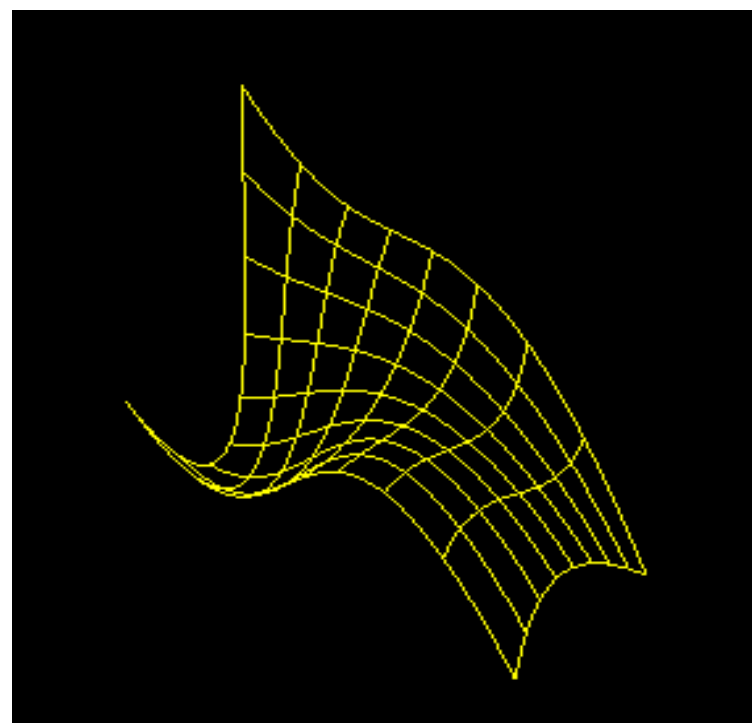
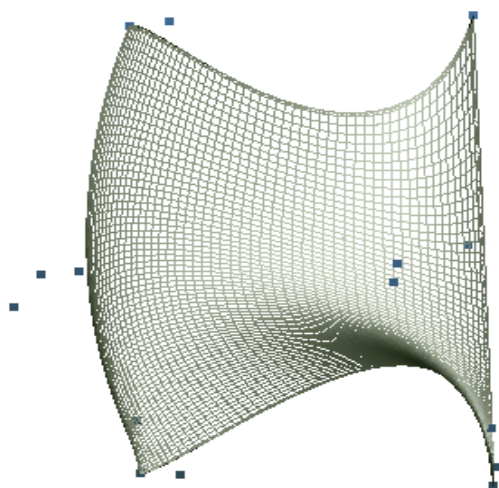


双三次Bezier曲面($m=n=3$)

$$p(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 P_{i,j} BEN_{i,3}(u) BEN_{j,3}(v) \quad (u, v) \in [0, 1] \times [0, 1]$$



图：双三次Bezier曲面及其控制网格





- Bezier Surfaces by matrix representation

$$P(u, v) = \begin{bmatrix} B_{0,n}(u) & B_{1,n}(u) & \cdots & B_{m,n}(u) \end{bmatrix} \begin{bmatrix} P_{00} & P_{01} & \cdots & P_{0m} \\ P_{10} & P_{11} & \cdots & P_{1m} \\ \cdots & \cdots & \cdots & \cdots \\ P_{n0} & P_{n1} & \cdots & P_{nm} \end{bmatrix} \begin{bmatrix} B_{0,m}(v) \\ B_{1,m}(v) \\ \cdots \\ B_{n,m}(v) \end{bmatrix}$$

上式的曲面称为 $m \times n$ 次的，在一般应用中， n 、 m 不大于 4。

• Properties of rectangular (矩形域Bezier 曲面性质)

—Bezier Surface hold similar properties of Bezier curves :

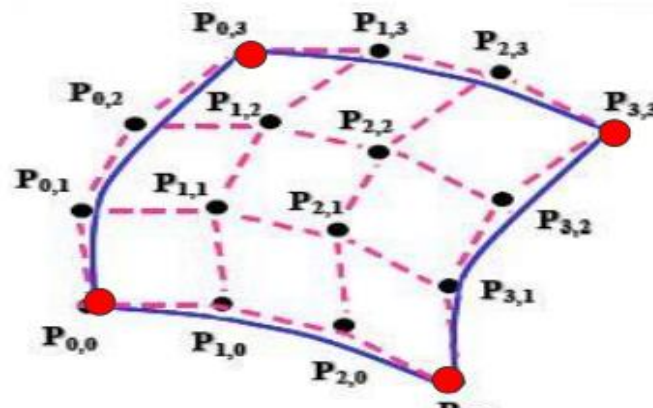
(1) Bezier 曲面特征网格的四个角点正好是Bezier 曲面的四个角点，即：

$$P(0,0) = P_{00}$$

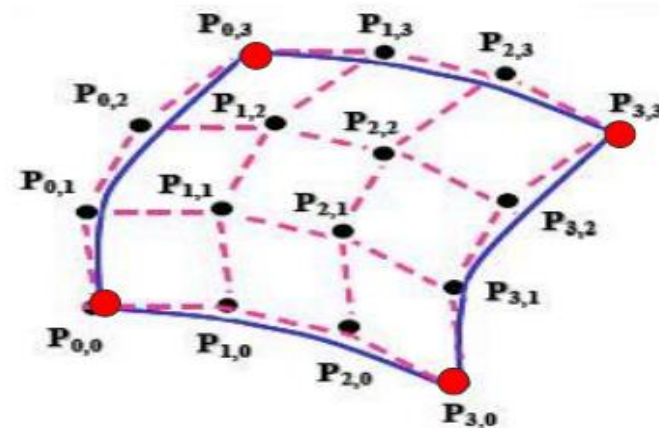
$$P(1,0) = P_{m0}$$

$$P(0,1) = P_{0n}$$

$$P(1,1) = P_{mn}$$



(2) Bezier 曲面特征网格最外一圈顶点定义Bezier 曲面的四条边界；



(3) 几何不变性

(4) 对称性

(5) 凸包性



- Geometric continuity (几何连续性) (了解)
用于**Bezier**曲面片的拼接

- De Casteljau algorithm of **Bezier** curves can be extended for surfaces. (了解)



B-Spline Curves and surfaces

Why to introduce B-Spline (B样条)



- **Bezier curve/surface has many advantages, but they have two main shortcomings:**

- Bezier curve/surface cannot be modified locally.

(不能作局部修改，牵一发而动全身，因为每个Bernstein多项式在整个区间 $[0,1]$ 上都有支撑，所以每个控制点对0到1之间的 t 值处的曲线都有影响)

- It is very complex to satisfy geometric continuity conditions for Bezier curves or surfaces joining.

Bezier曲线/曲面拼接时，满足几何连续性条件是十分困难的



• History of B-splines

- In 1946, Schoenberg proposed a spline-based method to approximate curves.
- It's motivated by runge-kutta problem in interpolation: high degree polynomial may surge upper and down
高阶多项式容易产生不稳定的上下抖动
- Why not use lower degree piecewise polynomial with continuous joining? 用分段低阶多项式通过连续的连接(分段连续多项式的方法)来逼近高阶多项式,既克服了波动现象,曲线也是低次的.
- that's Spline.
- 样条可以被认为是一个世纪最重要的发现,在很多工程应用中非常有用。



- But people thought it's impossible to use Spline in shape design, because complicated computation. (工程上的东西往往是比较简单，直观，好用，所以当时工程师认为这么复杂的东西不能用)
- In 1972, based on Schoenberg's work, 数学家 Gordon and Riesenfeld introduced “B-Spline” and lots of corresponding geometric algorithms. 开始在工业界得到广泛应用
- B-Spline retains all advantages of Bezier curves, and overcomes the shortcomings of Bezier curves.
- Similar to Bezier Curve using Bernstein basis functions, B-Spline curves **uses B-Spline basis functions.**

B-Spline curves and its Properties



$$P(t) = \sum_{i=0}^n P_i B_{i,n}(t), t \in [0,1]$$

- Formula of B-Spline Curve.

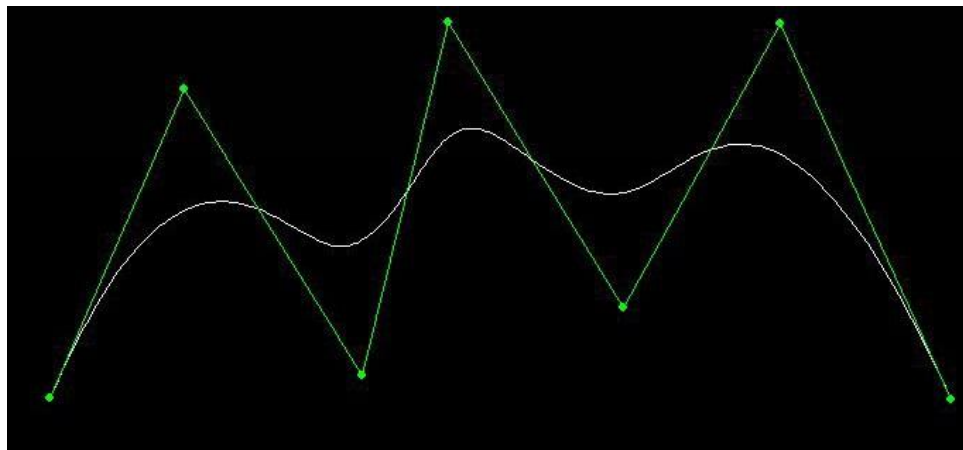
$$P(t) = \sum_{i=0}^n P_i N_{i,k}(t) \quad t \in [t_{k-1}, t_{n+1}]$$

$$P(u) = \sum_{i=0}^n P_i B_{i,k}(u)$$
$$u \in [u_{k-1}, u_{n+1}]$$

- $P_i (i = 0, 1, \dots, n)$ are control points.
- $N_{i,k}(t)$ ($i=0,1,\dots,n$) are the i -th B-Spline basis function of order k . (第 i 个 k 阶 ($k-1$ 次) B样条基函数, 对 Bezier 曲线来说, 阶数 = 次数; 对 B 样条来说, 阶数是次数+1)
- B-Spline basis function is a order k (degree $k-1$) piecewise polynomial (k 阶分段多项式) determined by the knot vector, which is a non-decreasing set of numbers (非减实数序列).
B样条基函数是 k 阶分段多项式, 它们由节点向量唯一决定.



- Demo of B-spline



- The story of order & degree
 - G Farin: degree, Computer Aided Geometric Design
 - Les Piegl: order, Computer Aided Design

B-Spline Basis Function

$$P(t) = \sum_{i=0}^n P_i N_{i,k}(t) \quad t \in [t_{k-1}, t_{n+1}]$$

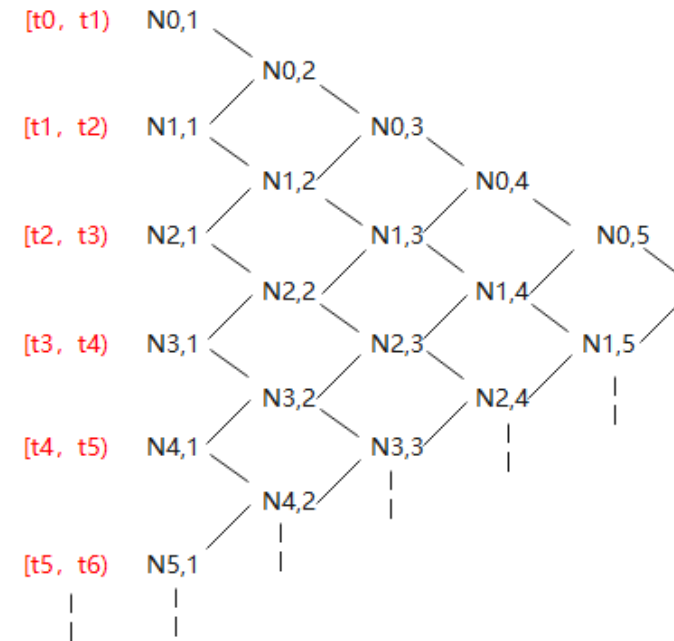
• Definition of B-Spline Basis Function

– de Boor-Cox recursion formula:

由 1 阶构造 2 阶, 2 阶构造 3 阶, 3 阶构造 4 阶...

$$N_{i,1}(t) = \begin{cases} 1 & t_i < t < t_{i+1} \\ 0 & \text{Otherwise} \end{cases}$$

$$N_{i,k}(t) = \frac{t - t_i}{t_{i+k-1} - t_i} N_{i,k-1}(t) + \frac{t_{i+k} - t}{t_{i+k} - t_{i+1}} N_{i+1,k-1}(t)$$

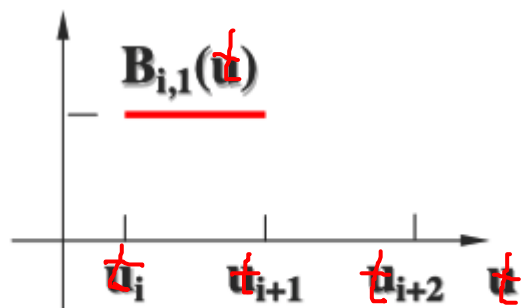


– Knot Vector: a sequence of non-decreasing number

$$t_0, t_1, \dots, t_{k-1}, t_k, \dots, t_n, t_{n+1}, \dots, t_{n+k-1}, t_{n+k}$$

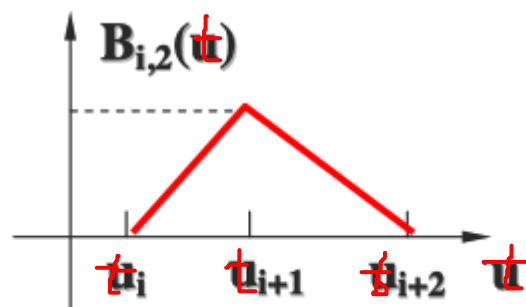
并约定 $\frac{0}{0} = 0$ 。

递推公式表明：欲确定第 i 个 k 阶 B 样条 $N_{i,k}(t)$ 需要用到 $t_i, t_{i+1}, \dots, t_{i+k}$ 共 $k+1$ 个节点，称区间 $[t_i, t_{i+k}]$ 为 $N_{i,k}(t)$ 的支撑区间。曲线方程中， $n+1$ 个控制节点 P_i 要用到 $n+1$ 个 k 阶 B 样条基 $N_{i,k}(t)$ 。它们的支撑区间的并集定义了这一组 B 样条基的节点矢量 $T = [t_0, t_1, \dots, t_{n+k}]$ 。



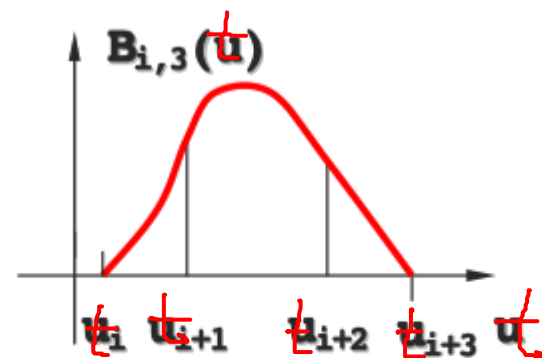
(1阶B样条)

https://blog.csdn.net/qq_41552508



(2阶B样条)

https://blog.csdn.net/qq_41552508



$B_{i,3}(u)$ 的图像

https://blog.csdn.net/qq_41552508

$$N_{i,1}(t) = \begin{cases} 1 & t_i < t < t_{i+1} \\ 0 & \text{Otherwise} \end{cases}$$

$$N_{i,k}(t) = \frac{t - t_i}{t_{i+k-1} - t_i} N_{i,k-1}(t) + \frac{t_{i+k} - t}{t_{i+k} - t_{i+1}} N_{i+1,k-1}(t)$$

B-Spline Basis Function

$$P(t) = \sum_{i=0}^n P_i N_{i,k}(t) \quad t \in [t_{k-1}, t_{n+1}]$$

- Questions:

$$N_{i,k}(t) \begin{cases} \geq 0 & t \in [t_i, t_{i+k}] \\ = 0 & \text{otherwise} \end{cases}$$

- What is nonzero domain(非零区间) of B-Spline basis function $N_{i,k}(t)$?

$$t_0, t_1, \dots, t_{k-1}, t_k, \dots, t_n, t_{n+1}, \dots, t_{n+k-1}, t_{n+k}$$

- How many knots does it need?

- What is the definition domain of the curves?

B样条插值出的曲线的定义区间是什么?

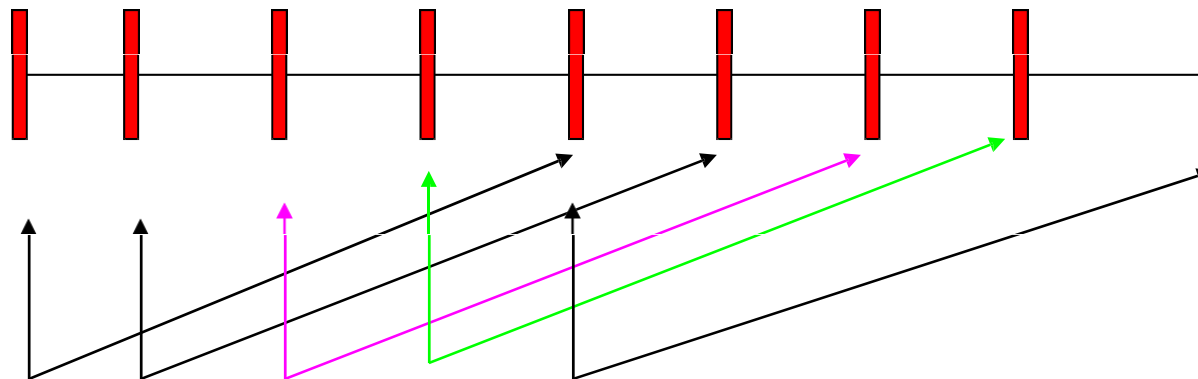


B-Spline Basis Function

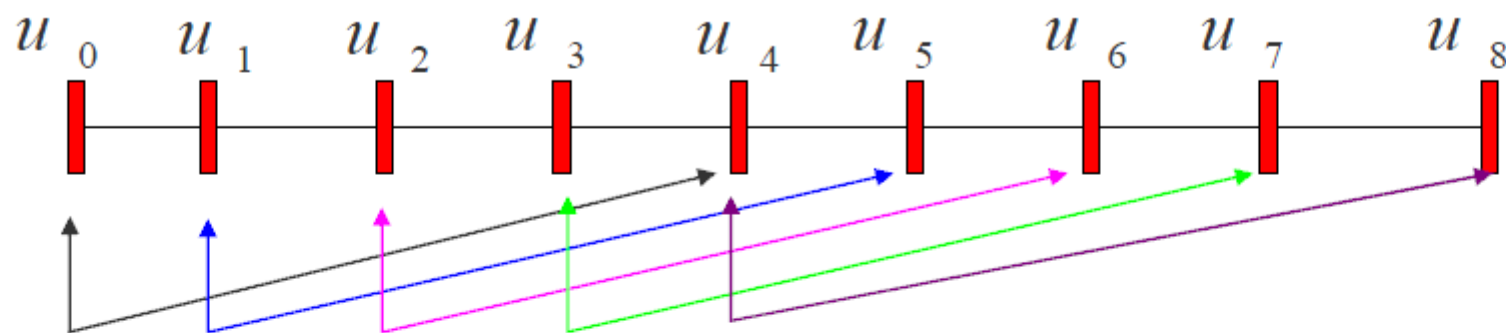
- take $k=4$, $n=4$ as example

$$P(u) = \sum_{i=0}^4 P_{i,4}(u)$$

$u_0, u_1, u_2, u_3, u_4, u_5, u_6, u_7, u_8$



$$P(t) = \sum_{i=0}^n P_{i,k}(t) \quad t \in [t_{k-1}, t_{n+1}]$$

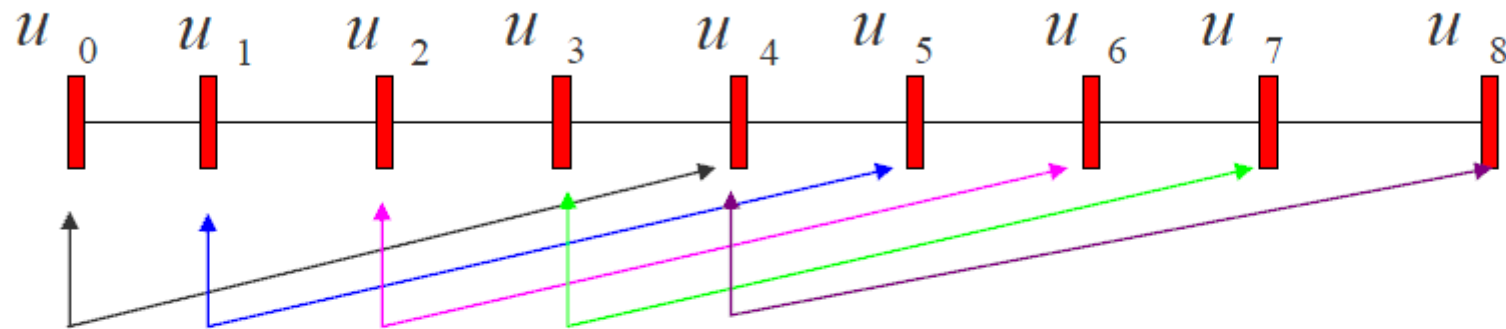


$P_0B_{0,4}(u)$ ，涉及 u_0 到 u_4 个节点 $P_3B_{3,4}(u)$ ，涉及 u_3 到 u_7 个节点
 $P_1B_{1,4}(u)$ ，涉及 u_1 到 u_5 个节点 $P_4B_{4,4}(u)$ ，涉及 u_4 到 u_8 个节点
 $P_2B_{2,4}(u)$ ，涉及 u_2 到 u_6 个节点

– 局部支持性：

- 区间 $t \in [t_i, t_{i+1}]$ 上的曲线仅由至多 k 个控制点 (control point) $P_j (j = i - k + 1, \dots, i)$ 决定。
- 修改控制点 P_i 仅会影响到区间 (t_i, t_{i+k}) 上的曲线。

“阶数+顶点”等于节点向量的个数。问函数的定义区间？



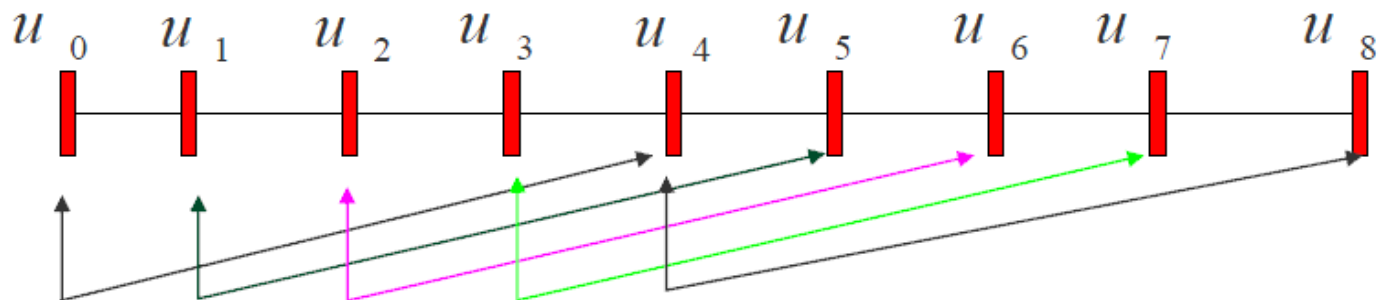
区间要合法，区间里必须要有足够的基函数与顶点配对

哪个区间是第一个开始有意义的区间呢？

上图区间是 u_3 到 u_5 （从 u_{k-1} 到 u_{n+1} ），B样条基函数严重依

赖于节点向量的分布

B样条曲线所对应的节点向量区间



上述曲线被分为两段， $u_3 u_4$ ， $u_4 u_5$ ， P_0, P_1, P_2, P_3 控制第一段， P_1, P_2, P_3, P_4 控制第二段。

上面的曲线被分成两段： $u_3 u_4$ ， $u_4 u_5$ 。如有5个顶点 P_0 、 P_1 、 P_2 、 P_3 、 P_4 ，B样条是一段段过渡过去

哪些基函数是在 $u_3 u_4$ 区间里有定义？正好是 $P_0 P_1 P_2 P_3$

$P_1 P_2 P_3 P_4$ 在 $u_4 u_5$ 区间里有定义，两端之间有三个顶点是一样的，这样就保证了两段拼接的效果非常好

- 哪些基函数在 $[u_{k-1}, u_k]$ 区间有定义？该区间正好 $P_0, P_1, \dots, P_{k-1}$ 共 k 个控制点生效。
- 哪些基函数在 $[u_k, u_{k+1}]$ 区间有定义？该区间正好 $P_1, P_2, \dots, P_k$ 共 k 个控制点生效。
-
- 哪些基函数在 $[u_n, u_{n+1}]$ 区间有定义？该区间正好 $P_{n-k+1}, \dots, P_n$ 共 k 个控制点生效。
- 每个小区间最多 k 个控制点影响，这些连续区间的每个区间正好由 k 个控制点影响。

B样条曲线所对应的节点向量区间： $u \in [u_{k-1}, u_{n+1}]$

因为在这些区间才有足够多的非0基函数（ k 个）在该区间有定义（对该区间支持）。



B样条曲线定义:

$$P(u) = \sum_{i=0}^n P_i B_{i,k}(u) \quad B_{i,1}(u) = \begin{cases} 1 & u_i < x < u_{i+1} \\ 0 & \text{Otherwise} \end{cases}$$
$$u \in [u_{k-1}, u_{n+1}] \quad B_{i,k}(u) = \frac{u - u_i}{u_{i+k-1} - u_i} B_{i,k-1}(u) + \frac{u_{i+k} - u}{u_{i+k} - u_{i+1}} B_{i+1,k-1}(u)$$

u_i 是节点值, $U = (u_0, u_1, \dots, u_{n+k})$ 构成了 k 阶 ($k-1$ 次) B样条函数的节点矢量

B样条曲线所对应的节点向量区间: $u \in [u_{k-1}, u_{n+1}]$



B-Spline Basis Function

- Properties:

- Non-negativity and local support 非负性和局部支撑性

- $N_{i,k}(t)$ is non-negative
- $N_{i,k}(t)$ is a non-zero polynomial on $[t_i, t_{i+k}]$

$$N_{i,k}(t) \begin{cases} \geq 0 & t \in [t_i, t_{i+k}] \\ = 0 & \text{otherwise} \end{cases}$$

- Partition of Unity 归一性

- The sum of all non-zero order k basis functions on $[t_{k-1}, t_{n+1}]$ is 1.

该区间内所有 k 阶分段非零基函数的和为 1。

$$\sum_{i=0}^n N_{i,k}(t) = 1 \quad t \in [t_{k-1}, t_{n+1}]$$



B-Spline Basis Function

- Properties:
 - Differential equation of the basis function: 基函数所满足的微分方程

$$N'_{i,k}(t) = \frac{k-1}{t_{i+k-1} - t_i} N_{i,k-1}(t) + \frac{k-1}{t_{i+k} - t_{i+1}} N_{i+1,k-1}(t)$$

- Please compare with the Bernstein base:

$$B'_{i,n}(t) = n[B_{i-1,n-1}(t) - B_{i,n-1}(t)],$$
$$i = 0, 1, \dots, n;$$



B-Spline

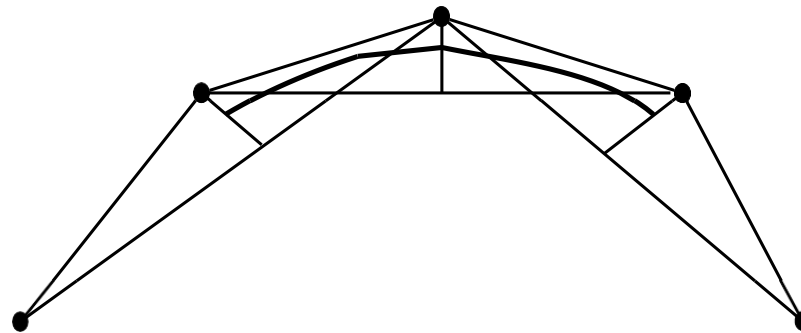
- Category(分类) of B-Spline
 - ❑ General Curves could be categorized to two groups by checking if the start point and the endpoint are overlapped:根据其起始点和终止点是否重叠来进行分类
 - Open Curves 不重合: 开曲线
 - Close Curves 重合: 闭曲线
 - ❑ According to the distribution(分布) of the knots in knot vector节点向量, B-Spline could be classified to the following four groups:



Uniform B-Spline(均匀B样条)

– (1) Uniform B-Spline

- The knots are uniform distributed 节点成等差数列均分排布, like $0, 1/8, 2/8, 3/8, 4/8, 5/8, 6/8, 7/8, 1$
- This kind of knot vector defines uniform B-Spline basis function



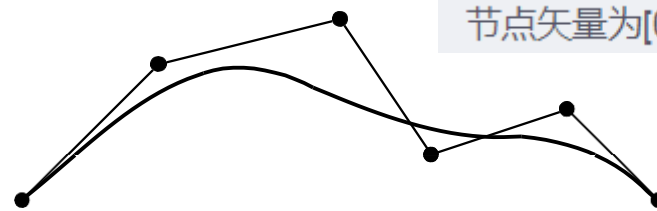
uniform B-Spline of Degree 3 (3次均匀的B样条曲线)

Quasi-Uniform B-Spline(准均匀B样条)



– (2) Quasi-Uniform B-Spline

- Different from uniform B-Spline, it has:
 - the start-knot and end-knot have repetitiveness(重复度) of k .
准均匀B样条的起始节点和终止节点都具有 k 的重复度
 - Uniform B-Spline does not retain the “end point” property of Bezier Curve, which means the start point and end point of uniform B-Spline are no-longer the same as the start point and end point of the control points. However, quasi-Uniform B-Spline retains this “end point” property



节点矢量为 $[0, 0, 0, 0, 1/3, 2/3, 1, 1, 1, 1]$

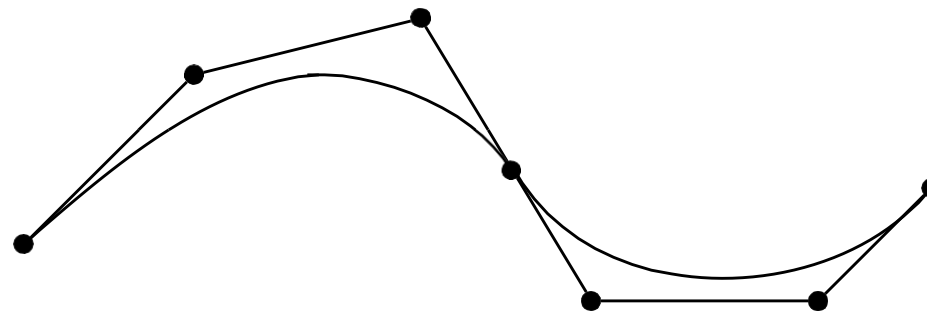
Quasi-uniform B-Spline curve of degree 3
(准均匀三次B样条曲线)

Piecewise Bezier Curve(分段Bezier曲线)



– (3) Piecewise Bezier Curve

- the start-knot and end-knot have repetitiveness(重复度) **of k**
 - all other knots have repetitiveness of $k-1$
 - Then each curve segment will be Bezier curves
- 这样，所有的曲线段都是Bezier曲线



Piecewise B-Spline Curve of degree 3
(三次分段Bezier 曲线)

图中有7个控制顶点, $n=6$, 阶数为4, 因此节点数为 $6+4+1=11$, 节点矢量为 $T = [t_0, t_1, \dots, t_{10}] = [0, 0, 0, 0, 0.5, 0.5, 0.5, 1, 1, 1, 1]$



Piecewise Bezier Curve(分段Bezier曲线)

- For piecewise Bezier curve, the different pieces of the curve are relatively independent. Moving the control point will only influence the corresponding piece of curve, while other pieces of curves will be not change. Furthermore, the algorithms for Bezier could also be used for piecewise Bezier Curve.

对于分段Bezier曲线，不同的曲线段相对独立。移动控制点只会影响其所在的Bezier曲线段，而其他的Bezier曲线段都不会改变。甚至所有关于Bezier曲线的算法可以同样地适用于分段Bezier曲线。

- But this method need more data to define the curve (more control points, more knots).

Non-uniform B-Spline (非均匀B样条)



– (4) Non-uniform B-Spline

- The knot vectors $T = [t_0, t_1, \dots, t_{n+k}]$ satisfy conditions that the sequence of knots is non-decreasing(非递减).
 - repetitiveness (重复度) of two end knots, $\leq k$
 - repetitiveness (重复度) of other knots, $\leq k-1$
- This kind of knot vector defines the non-uniform B-Spline.



Properties of B-Spline Curves

$$P(t) = \sum_{i=0}^n P_i N_{i,k}(t)$$

- Properties of B-Spline curves

- Local(局部性) 基函数 $N_{i,k}(t)$ 在 $[t_i, t_{i+k})$ 上非零;

- The curve in interval $t \in [t_i, t_{i+1}]$ is only affected by at most k control points. $P_j (j = i - k + 1, \dots, i)$ and is independent of other control points.
- Changing the position of control point P_i will only affect the curve on interval (t_i, t_{i+k})

? 改变一条以 $P_0, P_1, \dots, P_9$ 为控制顶点的4阶B样条曲线的一个顶点 P_5 , 有几段曲线的形状会改变?

P_5 影响在区间 $[t_5, t_9) \in [t_3, t_{10}]$ 上的曲线, 影响了4段曲线 $[t_5, t_6)$ 、 $[t_6, t_7)$ 、 $[t_7, t_8)$ 、 $[t_8, t_9)$



Properties of B-Spline

- Piecewise polynomial (分段多项式)
在任何一个由相邻节点确定的区间 (knot span) 上, $P(t)$ 是一个关于 t 的次数不超过 k 的多项式。
- Continuity(连续性)
- Convex hull(凸包性)
- Derivative formula(导数公式)
- Variation Diminishing Property(变差缩减性)
- Geometry invariability(几何不变性)
- Affine invariability(仿射不变性)
- Line holding(直线保持性)
- Flexibility(灵活性)



De Boor Algorithm

- To compute $P(t)$ (a point on the curve), you could use B-Spline formula, but it is more efficient to use de Boor algorithm.
- De Boor Algorithm:

→ 由 $P_{j-k+1} \sim P_j$ 共 k 个控制点影响
先将 t 固定在区间 $[t_j, t_{j+1})$ ($k-1 \leq j \leq n$) 上, 由 de Boor-Cox 公式有

$$\begin{aligned} P(t) &= \sum_{i=0}^n P_i N_{i,k}(t) = \sum_{i=j-k+1}^j P_i N_{i,k}(t) \\ &= \sum_{i=j-k+1}^j P_i \left[\frac{t-t_i}{t_{i+k-1}-t_i} N_{i,k-1}(t) + \frac{t_{i+k}-t}{t_{i+k}-t_{i+1}} N_{i+1,k-1}(t) \right] \\ &= \sum_{i=j-k+1}^j \left[\frac{t-t_i}{t_{i+k-1}-t_i} P_i + \frac{t_{i+k-1}-t}{t_{i+k-1}-t_i} P_{i-1} \right] N_{i,k-1}(t) \quad t \in [t_j, t_{j+1}] \end{aligned}$$

◆ 曲线 $P(t)$ 从 K 阶 B 样条到 $k-1$ 阶 B 样条表示的递推公式



De Boor Algorithm

- Let

$$P_i^{[r]}(t) = \begin{cases} P_i, r=0, i=j-k+1, j-k+2, \dots, j \\ \frac{t-t_i}{t_{i+k-r}-t_i} P_i^{[r-1]}(t) + \frac{t_{i+k-r}-t}{t_{i+k-r}-t_{i-1}} P_{i-1}^{[r-1]}(t), \\ r=1, 2, \dots, k-1; i=j-k+r+1, j-k+r+2, \dots, j \end{cases}$$

- Then

$$P(t) = \sum_{i=j-k+1}^j P_i N_{i,k}(t) = \sum_{i=j-k+2}^j P_i^{[1]}(t) N_{i,k-1}(t)$$

上式是一条曲线 $P(t)$ 从 k 阶到 $(k-1)$ 阶B样条表示的递推公式，反复利用此公式可得 $P(t) = P_j^{[k-1]}(t)$

- 此即为著名的de Boor算法



De Boor Algorithm

- The recursion of De Boor Algorithm is as shown below:

$$P_0$$

$$P_1$$

经过 1 轮递推后

经过 2 轮递推后

$$P_{j-k+1}$$

$$P_{j-k+2} \rightarrow P_{j-k+2}^{[1]}$$

$$P_{j-k+3} \rightarrow P_{j-k+3}^{[1]} \rightarrow P_{j-k+3}^{[2]}$$

经过 k-1 轮递推后

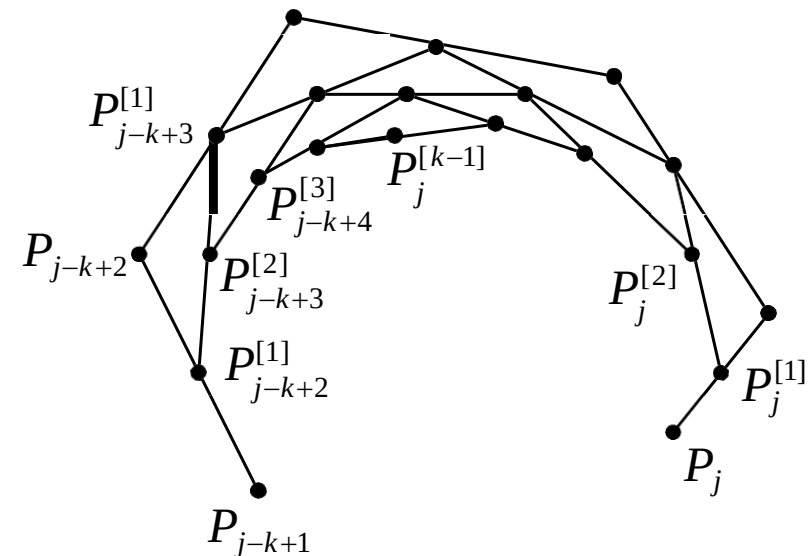
$$P_j \rightarrow P_j^{[1]} \rightarrow P_j^{[2]} \rightarrow \dots \rightarrow P_j^{[k-1]}$$

$$P_n$$



De Boor Algorithm

- Geometric meaning (几何意义) of De Boor Algorithm
 - It has an intuitive geometric interpretation: corner cutting (割角).
 - It means that using line segment $P_i^{[r]}P_{i+1}^{[r]}$ to cut corner $P_i^{[r-1]}$. Begin from polygon $P_{j-k+1}P_{j-k+2} \cdots P_j$, after $k-1$ steps of cutting, we finally get point $P_j^{[r-1]}(t)$ on the curve $P(t)$.





B-Spline Surface(B样条曲面)

- Given knot vectors in axes(参数轴) U and V:

$$U = [u_0, u_1, \dots, u_{m+p}]$$

$$V = [v_0, v_1, \dots, v_{n+q}]$$

- B-Spline surface of order $p \times q$ is defined as:

$$P(u, v) = \sum_{i=0}^m \sum_{j=0}^n P_{ij} N_{i,p}(u) N_{j,q}(v)$$

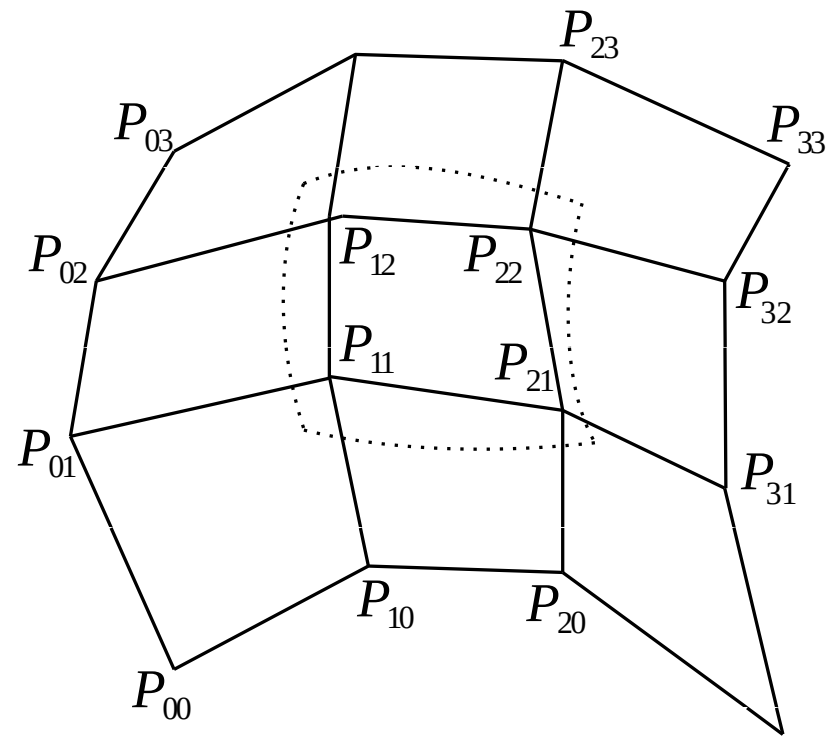


B-Spline Surface

- P_{ij} are the control points of B-Spline Surface, which are usually referred to as the control net (控制网格、特征网格).
- $N_{i,p}(u)$ and $N_{j,q}(v)$ are B-Spline basis functions of order p and q , one for each direction (U and V), which could also be computed by de Boor-Cox formula.



B-Spline Surface



An example: B-Spline P_{30}
surface of degree $3*3$ (双三
次B样条曲面片)



思考题

- 由 k 个控制顶点 $P_i (i=1, \dots, k)$ 所决定的 n 次B样条曲线，由____段 n 次B样条曲线段光滑连接而成。(思考定义域多少，有几段连接)
A) $k-n-2$ B) $k-n-1$ **C) $k-n$** D) $k-n+1$

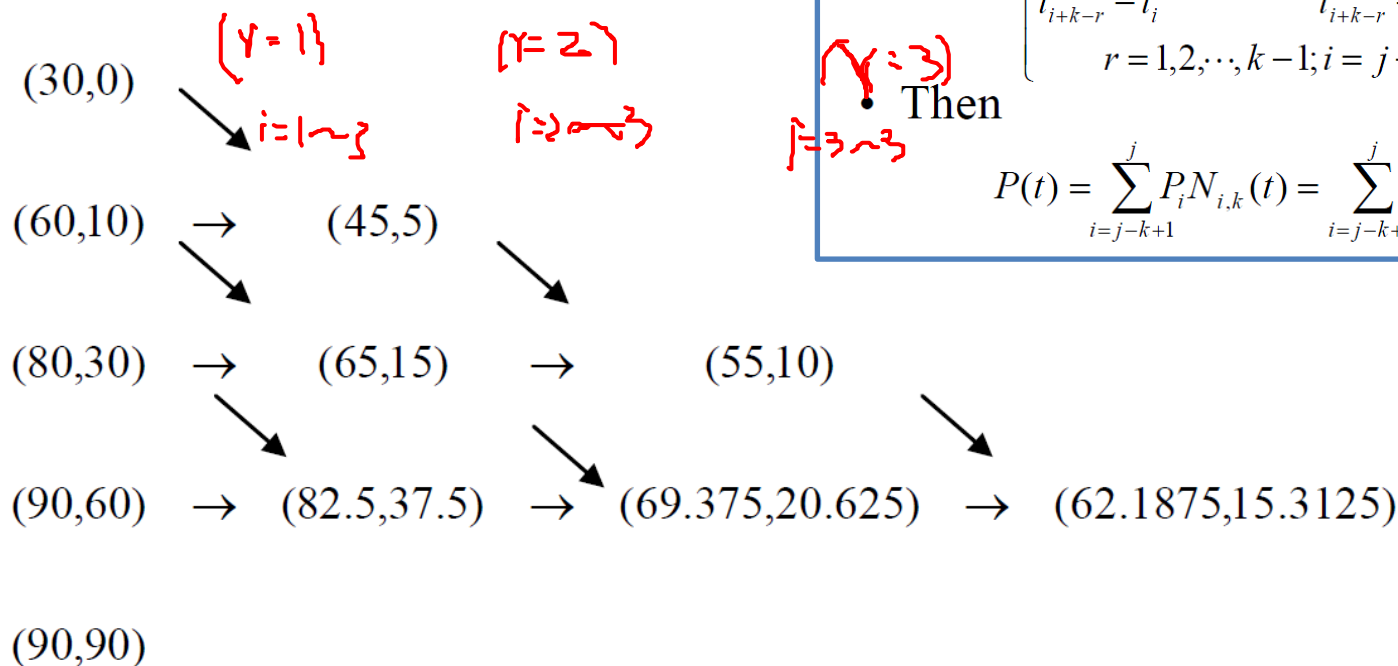
- 五个控制顶点的三次B样条的节点向量应该由几个节点构成？
a. 5 b. 7 c. 8 **d. 9**

- 一条以 p_0, p_1, p_2, p_3, p_4 为控制顶点的4阶(三次)B样条曲线，其节点向量为 $\{0, 0, 0, 1, 2, 3, 4, 4, 4\}$ ，则其定义域为 **(1, 3)**

用 de Boor 算法, 求以(30,0),(60,10),(80,30),(90,60),(90,90)为控制顶点、以 $T=(0,0,0,0,0.5,1,1,1,1)$ 为节点向量的三次 B 样条曲线在 $t=1/4$ 处的值。

先将 t 固定在区间 $[t_j, t_{j+1})$ ($k-1 \leq j \leq n$) 上, $\{t_3, t_4\}$
 p_0, p_1, p_2, p_3

$$\begin{cases} r=0 \\ j=0 \sim 3 \end{cases}$$



• Let

$$P_i^{[r]}(t) = \begin{cases} P_i, r=0, i=j-k+1, j-k+2, \dots, j \\ \frac{t-t_i}{t_{i+k-r}-t_i} P_i^{[r-1]}(t) + \frac{t_{i+k-r}-t}{t_{i+k-r}-t_i} P_{i-1}^{[r-1]}(t), \\ r=1, 2, \dots, k-1; i=j-k+r+1, j-k+r+2, \dots, j \end{cases}$$

Then

$$P(t) = \sum_{i=j-k+1}^j P_i N_{i,k}(t) = \sum_{i=j-k+2}^j P_i^{[1]}(t) N_{i,k-1}(t)$$

即 B 样条曲线在 $t=1/4$ 处的值为 (62.1875,15.3125)